High-Performance FPGA Accelerator for the Post-quantum Signature Scheme CROSS

Patrick Karl[1], Francesco Antognazza[2], Alessandro Barenghi[2], Gerardo Pelosi[2], and Georg Sigl[1,3]

[1] TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany, {patrick.karl,sigl}@tum.de
[2] Politecnico di Milano, Department of Electronics, Information and Bioengineering (DEIB), Milan, Italy, {francesco.antognazza,alessandro.barenghi,gerardo.pelosi}@polimi.it
[3] Fraunhofer Institute for Applied and Integrated Security, Garching, Germany, georg.sigl@aisec.fraunhofer.de

Abstract. In October 2024, the National Institute of Standards and Technology announced the second round candidates of its standardization effort for additional post-quantum signatures. One of these candidates is CROSS, a code-based scheme relying on the restricted syndrome decoding problem. In this work, we present the first hardware design of CROSS, delineating efficient implementation strategies for the critical components of the cryptographic scheme. Our architecture parallelizes rejection sampling in two dimensions, enabling to simultaneously generate multiple vectors, as well as multiple entries of these vectors. We implement hardware friendly modular reduction circuits requiring only shifts and additions to obtain a DSP-free design, and carefully schedule operations enabling to hide them behind more computationally intensive tasks. Depending on the chosen security level, our design generates a key pair in 8 to 148 μs, signs a message in 338 μs to 4.62 ms, and verifies a signature in 303 μs to 3.27 ms on a Xilinx Artix-7 device. We show that our design is among the fastest and smallest when compared with other on-ramp candidates, namely LESS, MEDS, MAYO, Raccoon and SDitH, and comparable to current standard-selected ML-DSA, FN-DSA, and SLH-DSA in terms of efficiency.

Keywords: Hardware Accelerator · CROSS · Post-quantum Digital Signature · Restricted Syndrome Decoding Problem · FPGA

1 Introduction

Post-quantum Cryptography (PQC) aims at designing cryptographic algorithms that are secure against adversaries equipped with both classical and quantum computers. These algorithms are based on mathematical frameworks encompassing algebraic lattices, error correction codes, isogeny maps between elliptic curves, iterative hash computations and multivariate quadratic equations. Their security relies on computational problems proven to be hard to solve, or strongly believed to be as such even against the computational advantages provided by a large quantum computer. In response to the advancements of quantum computing technologies, the National Institute of Standards and Technology (NIST) initiated the public Post-quantum Cryptography Standardization Process in December 2016 [oSU25]. The objective of this initiative was to select quantum-resistant public-key cryptographic algorithms for standardization. After four rounds of evaluation, NIST announced that the Key Encapsulation Mechanisms (KEMs) selected for standardization were CRYSTALS-Kyber (ML-KEM [oSU24b]), based on structured lattice problems and Hamming-Quasi-Cyclic (HQC) as the non-lattice-based KEM alternative.

The selection for digital signatures elected CRYSTALS-Dilithium (ML-DSA [oSU24a]) and Falcon [FHK+20] (FN-DSA), as structured lattice-based signatures, while selecting SPHINCS+ (SLH-DSA [oSU24c]) provides an alternative not relying on structured lattice problems.

While the current performance profile and space requirements of KEMs were deemed satisfactory for general purpose use by NIST, and a fallback in case cryptanalytic breakthroughs take place on lattice-based problems is available, the situation on the chosen signature schemes is different. Indeed, while SPHINCS+ provides a high-confidence fallback for CRYSTALS-Dilithium and Falcon, its high computational demands and relatively large signature size were considered to be hindering its general purpose use by NIST. As a consequence, in September 2022, NIST called for additional digital signature proposals (colloquially, "on-ramp"), providing better performances than SPHINCS+, while still relying on computationally hard problems not stemming from structured lattices. After a first evaluation round, NIST selected only 14 out of 40 submitted designs as candidates for further assessment in October 2024 [MMP+24]. Among the proposals, the Codes and Restricted Objects Signature Scheme (CROSS) [BBB+25a], based on the difficulty of the restricted syndrome decoding problem, was designed with the intent of being deployable as a general purpose, fast signature scheme.

To evaluate the candidates with respect to performance and applicability, benchmarking them on different software-, but also hardware platforms is crucial for the standardization process. NIST explicitly mentioned as targets for performance benchmarking either high-performance software implementations (for which NIST asked for the use of Intel AVX2 vector instructions) and efficient hardware implementations for which NIST recommended the AMD/Xilinx Artix-7 family of FPGAs in the past[1]. Artix-7 devices were chosen due to their low cost and satisfactory performance: indeed it is not uncommon to employ them where the cryptographic agility provided by the reconfigurability of an FPGA must be matched by competitive costs. While the hardware design and evaluation of the current candidates in the on-ramp is in its early stages, several works proposed designs for standalone accelerators described at the Register Transfer Level (RTL). In particular, RTL designs are available for the code-based schemes LESS [BWMG23] and MEDS [DLN+25], the multivariate scheme MAYO [SMA+24, HSK+24], the lattice-based Raccoon [NKZ+25] and the Multi-Party Computation-in-the-Head (MPCitH) proposal SDitH [DHSY24]. Furthermore, a HW/SW co-design approach for the MPCitH-based scheme MiRitH was introduced in [STW24].

Contributions. In this work, we contribute to the evaluation process by presenting the first hardware design for CROSS: it covers key generation, signature creation and verification, supports all 18 parameter sets (configurable at compile time) contained in the latest revision (v2.2) of the CROSS specification and is fully compatible with the C reference implementation in the NIST submission [BBB+25a]. Our contributions further include the following:

- We design an architecture that parallelizes rejection sampling in two dimensions, sampling 1) multiple vectors, and 2) multiple entries of each vector in parallel. We implement DSP-free modular reduction circuits by exploiting the special structure in the prime moduli used in CROSS employing a technique presented by Crandall [E93].
- We show how careful scheduling enables hiding the execution time of certain operations, leveraging parallelism to improve performance at negligible logic cost.
- We perform a Design Space Exploration (DSE) of our main building blocks to justify our parameter selection for performance/cost trade-offs within submodules.

[1] https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/cJxMq0_90gU/m/qbGEs3TXGwAJ, last access: October 22, 2025

- We provide a detailed comparison of our design with designs for signature schemes selected for standardization, as well as on-ramp candidates. We show that our performance is competitive to these designs while retaining moderate resource consumption. Moreover, we compare our results with optimized software implementations of CROSS for desktop-class processors as well as resource constrained microcontrollers.

The design goal of our implementation is minimizing the Area-Time (AT) product which we consider as an efficiency metric, and which is in line with the choice of an Artix-7 as a cost-optimized FPGA. We consequently focus on maximizing this metric by streamlining sampling and arithmetic operations and exploit parallelism as much as possible without duplicating major logic blocks. Moreover, our design is fully constant time, i.e. its runtime is independent of secret data.

2 Background

In this section, we introduce the notation used throughout the rest of the paper. We provide procedural descriptions of CROSS.KeyGen, CROSS.Sign and CROSS.Verify, briefly describe auxiliary primitives, and list the parameter sets. For more details, we refer to the corresponding specification [BBB+25a] and security guide [BBB+25b] of CROSS.

2.1 Fundamentals and Notation

Let $\lambda \in \{128, 192, 256\}$ be the security parameter. We denote scalars with lowercase letters (a), vectors with lowercase boldface letters ($\mathbf{v}$), matrices with uppercase letters (M), the support of algebraic groups as uppercase boldface letters ($\mathbf{E}$), and the finite field with p elements as $\mathbb{F}_p$, where p is a prime number. The computational problem of finding any error vector $\mathbf{e} \in \mathbb{F}_p^n$ such that $\mathbf{s} = \mathbf{e}H^\top$, given a uniformly random parity check matrix $H \in \mathbb{F}_p^{(n-k)\times n}$ and a non-null syndrome $\mathbf{s} \in \mathbb{F}_p^{n-k}$, is generally easy unless some constraints on the values of $\mathbf{e}$ are imposed. CROSS is based on a variant of the Syndrome Decoding Problem (SDP), where the restriction on $\mathbf{e}$ mandates that all its entries belong to a multiplicative subgroup $\mathbf{E}$ of $\mathbb{F}_p^*$ generated by a public element g of order z: $\langle g \rangle = \left\{ g^i \mid i \in \{1, \dots, z\} \right\} = \mathbf{E} \subset \mathbb{F}_p^*$, such that $\mathbf{e} \in \mathbf{E}^n$. This variant is known as *Restricted SDP* (RSDP). It is possible to build a bijection between elements of $\mathbf{E}^n$ and elements of $\mathbb{F}_z^n$: each entry of $\mathbf{e} \in \mathbf{E}^n$ can be expressed as $g^{\overline{x}}$ with $\overline{x} \in \mathbb{F}_z$. Denoting all objects in $\mathbb{F}_z$ with an overline, the vector $\overline{\mathbf{e}} = [\overline{e}_0, \dots, \overline{e}_{n-1}] \in \mathbb{F}_z^n$ corresponds to $\mathbf{e} = [e_0, \dots, e_{n-1}] = [g^{\overline{e}_0}, \dots, g^{\overline{e}_{n-1}}] \in \mathbf{E}^n$. For brevity, we denote $[g^{\overline{e}_0}, \dots, g^{\overline{e}_{n-1}}]$ as $g^{\overline{\mathbf{e}}}$.

For improved efficiency, CROSS considers also another restriction for $\mathbf{e}$: valid error vectors $\mathbf{e}$ are such that the corresponding $\overline{\mathbf{e}} \in \mathbb{F}_z^n$ is a codeword of a code of length n and dimension m, i.e. they are obtained by multiplying a vector $\overline{\mathbf{e}}_G \in \mathbb{F}_z^m$ by a generator matrix $\overline{M} \in \mathbb{F}_z^{m\times n}$. This version is called *RSDP with subgroup G* (RSDP(G)). Any element $\mathbf{e} \in \mathbf{G}$ can thus be compactly represented by its corresponding $\overline{\mathbf{e}}_G$, and computed as $\mathbf{e} = g^{\overline{\mathbf{e}}_G \overline{M}}$. In CROSS, g is constant and defined as $g = 2$ for RSDP and $g = 16$ for RSDP(G).

Auxiliary Primitives. CROSS uses multiple instances of a Cryptographically Secure PseudoRandom Number Generator (CSPRNG) and a cryptographic hash function (Hash). Both are realized by the extendable output function SHAKE [oSU15], being either SHAKE-128 or SHAKE-256 depending on the security level. To instantiate multiple, independent CSPRNGs and Hashes, CROSS adopts a simple *domain separation* strategy consisting of appending a distinct 16 bit integer to the SHAKE input. For simplicity, we omit the details on domain separation in the remainder of our work. The corresponding instances of the symmetric primitives are denoted as CSPRNG($\texttt{seed}, \mathcal{S}$), taking as input a binary string `seed` and outputting a pseudorandom element of the set $\mathcal{S}$, and Hash(`input`), using

Algorithm 1 CROSS.KeyGen

Require: None
Ensure: $\mathsf{sk} = \mathsf{seed}_{\mathsf{sk}} \in \{0,1\}^{2\lambda}$, $\mathsf{pk} = (\mathsf{seed}_{\mathsf{pk}} \in \{0,1\}^{2\lambda}, \mathbf{s} \in \mathbb{F}_p^{n-k})$

1: $\mathsf{seed}_{\mathsf{sk}} \xleftarrow{\$} \{0,1\}^{2\lambda}$ ▷ *get $seed_{sk}$ from system RNG*
2: $(\mathsf{seed}_{\mathbf{e}}, \mathsf{seed}_{\mathsf{pk}}) \leftarrow \mathrm{CSPRNG}(\mathsf{seed}_{\mathsf{sk}}, \{0,1\}^{2\lambda} \times \{0,1\}^{2\lambda})$ ▷ *expand $seed_{sk}$ into $seed_{\mathbf{e}}$, $seed_{\mathsf{pk}}$*
3: $\overline{W}, V^\top \leftarrow \mathrm{CSPRNG}(\mathsf{seed}_{\mathsf{pk}}, \mathbb{F}_z^{m\times(n-m)} \times \mathbb{F}_p^{k\times(n-k)})$ ▷ *sample matrices from $seed_{\mathsf{pk}}$*
$\overline{M} \leftarrow [\overline{W}, I_m]$
$V^\top \leftarrow \mathrm{CSPRNG}(\mathsf{seed}_{\mathsf{pk}}, \mathbb{F}_p^{k\times(n-k)})$
4: $H^\top \leftarrow [V^\top \mid I_{n-k}]$
5: $\overline{\mathbf{e}}_G \leftarrow \mathrm{CSPRNG}(\mathsf{seed}_{\mathbf{e}}, \mathbb{F}_z^m)$ ▷ *sample restricted error from $seed_{\mathbf{e}}$*
$\overline{\mathbf{e}} \leftarrow \overline{\mathbf{e}}_G \overline{M}$
$\overline{\mathbf{e}} \leftarrow \mathrm{CSPRNG}(\mathsf{seed}_{\mathbf{e}}, \mathbb{F}_z^n)$
6: $\mathbf{e} \leftarrow g^{\overline{\mathbf{e}}}$ ▷ *convert from $\mathbb{F}_z^n$ to $\mathbb{F}_p^n$*
7: $\mathbf{s} \leftarrow \mathbf{e}H^\top$ ▷ *compute syndrome*
8: **return** $(\mathsf{sk} = \mathsf{seed}_{\mathsf{sk}}, \mathsf{pk} = (\mathsf{seed}_{\mathsf{pk}}, \mathbf{s}))$;

the binary string `input`. In practice, $\mathrm{CSPRNG}(\mathtt{seed}, \mathcal{S})$ first generates pseudorandom bits via SHAKE and either outputs them directly (if $\mathcal{S}$ denotes a bitstring), or performs rejection sampling on the bits to produce finite field elements in $\mathbb{F}_p$, $\mathbb{F}_p^*$, or $\mathbb{F}_z$.

2.2 The CROSS Signature Scheme

The basis of CROSS is an interactive Zero-Knowledge Identification scheme called CROSS-ID. The verification key corresponds to an RSDP or RSDP(G) instance, i.e. a pair of parity check matrix H and syndrome $\mathbf{s}$, while the secret key is the solution to said instance, i.e. the restricted error vector $\mathbf{e}$ such that $\mathbf{s} = \mathbf{e}H^\top$. The CROSS-ID scheme enables a prover to show that it knows the solution to the problem instance without revealing the solution itself. In doing so, the prover uses its keypair instance to derive another random instance and shows the verifier either the solution, or the build process of the fresh random instance. Applying the Fiat-Shamir transform [FS86] to t parallel executions of this protocol yields the non-interactive signature scheme.

We now provide an overview of CROSS.KeyGen, CROSS.Sign and CROSS.Verify. Steps exclusive to the RSDP variant of CROSS are shown in teal, while the ones for RSDP(G) are in orange.

CROSS.KeyGen. Key generation (shown in Algorithm 1) starts by drawing the secret key bitstring $\mathsf{sk} = \mathsf{seed}_{\mathsf{sk}}$ from the system RNG (line 1), which is then expanded into the private $\mathsf{seed}_{\mathbf{e}}$ and the public $\mathsf{seed}_{\mathsf{pk}}$ bitstrings (line 2). The later is used to generate one (for RSDP) or two (for RSDP(G)) public matrices (line 3–4) directly in systematic form by sampling only the non-identity portions V (and $\overline{W}$ for RSDP(G)). The private $\mathsf{seed}_{\mathbf{e}}$ bitstring is used to sample the secret exponent-vector $\overline{\mathbf{e}}$ (line 5), which in turn is used to generate the restricted error vector $\mathbf{e}$ (line 6). Finally, the syndrome $\mathbf{s}$ is computed (line 7), and returned together with $\mathsf{seed}_{\mathsf{pk}}$ as the public key $\mathsf{pk} = (\mathsf{seed}_{\mathsf{pk}}, \mathbf{s})$.

CROSS.Sign. Signature generation (shown in Algorithm 2) initially expands the secret key $\mathsf{seed}_{\mathsf{sk}}$ following a similar procedure as KeyGen, while retaining all intermediate results except for the syndrome $\mathbf{s}$ (line 1). Then, two uniformly random bitstrings, `seed` and `salt` are drawn from the system RNG (line 2) and used by the function SeedLeaves to compute a sequence of t round seeds $\mathtt{seed}[i]$, each one being an input of the t repetitions of CROSS-ID (line 3). This expansion of `seed` and `salt` is performed through a technique known as *seed tree* [BKP20] (or GGM tree [GGM86]), which relies on building a binary tree of input-length-doubling CSPRNGs. The leaves of the tree are used as round seeds in the commitment phase (lines 4–11) to compute all pairs of commitments $\mathtt{cmt}_0[i]$ and $\mathtt{cmt}_1[i]$. This phase is the most computationally expensive task of signature generation,

Algorithm 2 CROSS.Sign

Require: $\mathtt{sk} = \mathtt{seed_{sk}} \in \{0,1\}^{2\lambda}$, $\mathtt{msg} \in \{0,1\}^*$
Ensure: $\mathtt{sig} = (\mathtt{salt} \in \{0,1\}^{2\lambda}, \mathtt{dig_{cmt}} \in \{0,1\}^{2\lambda}, \mathtt{dig_{chall_2}} \in \{0,1\}^{2\lambda}, \mathtt{path}, \mathtt{proof}, \mathtt{rsp})$

1: $\overline{\mathbf{e}}, \overline{\mathbf{e}}_G, H^\top, \overline{M} \leftarrow$ ExpandSK($\mathtt{seed_{sk}}$) | $\overline{\mathbf{e}}, H^\top \leftarrow$ ExpandSK($\mathtt{seed_{sk}}$)
2: $\mathtt{seed} \xleftarrow{\$} \{0,1\}^\lambda$; $\mathtt{salt} \xleftarrow{\$} \{0,1\}^{2\lambda}$ ▷ *get seed and salt from system RNG*
3: $(\mathtt{seed}[0], \ldots, \mathtt{seed}[t-1]) \leftarrow$ SeedLeaves($\mathtt{seed}, \mathtt{salt}$) ▷ *compute seed tree, return leaves as round seeds*
4: **for** $i \leftarrow 0$ **to** $t-1$ **do**
$\overline{\mathbf{e}}'_G[i], \mathbf{u}'[i] \leftarrow$ CSPRNG($\mathtt{seed}[i] \| \mathtt{salt}, \mathbb{F}_z^n \times \mathbb{F}_p^n$) ▷ *sample error $\overline{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$ from round seed*
5: $\overline{\mathbf{e}}'[i] \leftarrow \overline{\mathbf{e}}'_G[i]\overline{M}$ | $\overline{\mathbf{e}}'[i], \mathbf{u}'[i] \leftarrow$ CSPRNG($\mathtt{seed}[i] \| \mathtt{salt}, \mathbb{F}_z^n \times \mathbb{F}_p^n$)
$\overline{\mathbf{v}}_G[i] \leftarrow \overline{\mathbf{e}}_G - \overline{\mathbf{e}}'_G[i]$
6: $\overline{\mathbf{v}}[i] \leftarrow \overline{\mathbf{e}} - \overline{\mathbf{e}}'[i]$ ▷ *compute transform between error vectors*
7: $\mathbf{v}[i] \leftarrow g^{\overline{\mathbf{v}}[i]}$ ▷ *convert from $\mathbb{F}_z^n$ to $\mathbb{F}_p^n$*
8: $\mathbf{u}[i] \leftarrow \mathbf{v}[i] \odot \mathbf{u}'[i]$ ▷ *$\odot$: component-wise product*
9: $\mathbf{s}'[i] \leftarrow \mathbf{u}[i]H^\top$ ▷ *compute syndrome*
10: $\mathtt{cmt}_0[i] \leftarrow$ Hash($\mathbf{s}'[i] \| \overline{\mathbf{v}}_G[i] \| \mathtt{salt}$) | $\mathtt{cmt}_0[i] \leftarrow$ Hash($\mathbf{s}'[i] \| \overline{\mathbf{v}}[i] \| \mathtt{salt}$)
11: $\mathtt{cmt}_1[i] \leftarrow$ Hash($\mathtt{seed}[i] \| \mathtt{salt}$)
12: $\mathtt{dig_{cmt_0}}, \mathtt{dig_{cmt_1}} \leftarrow$ TreeRoot($\mathtt{cmt}_0[0] \| \ldots \| \mathtt{cmt}_0[t-1]$), Hash($\mathtt{cmt}_1[0] \| \ldots \| \mathtt{cmt}_1[t-1]$)
13: $\mathtt{dig_{cmt}} \leftarrow$ Hash($\mathtt{dig_{cmt_0}} \| \mathtt{dig_{cmt_1}}$) ▷ *compute hashes over commitments*
14: $\mathtt{dig_{msg}} \leftarrow$ Hash($\mathtt{msg}$) ▷ *$\cdots$ and message*
15: $\mathtt{dig_{chall_1}} \leftarrow$ Hash($\mathtt{dig_{msg}} \| \mathtt{dig_{cmt}} \| \mathtt{salt}$)
16: $\mathtt{chall}_1 \leftarrow$ CSPRNG($\mathtt{dig_{chall_1}}, (\mathbb{F}_p^*)^t$) ▷ *sample $\mathtt{chall}_1$ over multiplicative subgroup $\mathbb{F}_p^\star$*
17: **for** $i \leftarrow 0$ **to** $t-1$ **do**
18: $\mathbf{e}'[i] \leftarrow g^{\overline{\mathbf{e}}'[i]}$
19: $\mathbf{y}[i] \leftarrow \mathbf{u}'[i] + \mathtt{chall}_1[i]\mathbf{e}'[i]$ ▷ *compute first responses*
20: $\mathtt{dig_{chall_2}} \leftarrow$ Hash($\mathbf{y}[0] \| \ldots \| \mathbf{y}[t-1] \| \mathtt{dig_{chall_1}}$) ▷ *hash first responses*
21: $\mathtt{chall}_2 \leftarrow$ CSPRNG($\mathtt{dig_{chall_2}}, \mathcal{B}_{t,w}$) ▷ *$\mathcal{B}_{t,w}$: the set of length t bitstrings of weight w*
22: $\mathtt{proof} \leftarrow$ TreeProof($\mathtt{cmt}_0[0] \| \ldots \| \mathtt{cmt}_0[t-1], \mathtt{chall}_2$) ▷ *collect Merkle proof nodes*
23: $\mathtt{path} \leftarrow$ SeedPath($\mathtt{seed}, \mathtt{salt}, \mathtt{chall}_2$) ▷ *collect seed path nodes*
24: **for** $i \leftarrow 0$ **to** $t-1$ **do**
25: **if** $\mathtt{chall}_2[i] = 0$ **then**
26: $\mathtt{rsp}[i]_0 \leftarrow (\mathbf{y}[i], \overline{\mathbf{v}}_G[i])$ | $\mathtt{rsp}[i]_0 \leftarrow (\mathbf{y}[i], \overline{\mathbf{v}}[i])$ ▷ *pack tuples*
27: $\mathtt{rsp}[i]_1 \leftarrow \mathtt{cmt}_1[i]$
28: **return** $\mathtt{sig} = (\mathtt{salt}, \mathtt{dig_{cmt}}, \mathtt{dig_{chall_2}}, \mathtt{path}, \mathtt{proof}, \mathtt{rsp})$

where the original RSDP/RSDP(G) problem instance contained in the keypair is used to derive another random instance $\mathbf{s}'[i] = \mathbf{u}[i]H^\top$. The first commitment $\mathtt{cmt}_0[i]$ is computed from $\mathbf{s}'[i]$, while the second commitment $\mathtt{cmt}_1[i]$ binds to the generation of the new random problem instance via the digest of $\mathtt{seed}[i]$ and $\mathtt{salt}$, as all the elements of iteration i are derived deterministically from it. After computing all $\mathtt{cmt}_0[i]$ and $\mathtt{cmt}_1[i]$ values for the t parallel iterations, they are hashed into two digests, $\mathtt{dig_{cmt_0}}$ and $\mathtt{dig_{cmt_1}}$ (line 12): all $\mathtt{cmt}_1[i]$ are simply concatenated and hashed, whereas $\mathtt{dig_{cmt_0}}$ is the Merkle tree root obtained via the TreeRoot function when the tree leaves are $\mathtt{cmt}_0[i]$. The first challenge $\mathtt{chall}_1$ is generated from the seed material $\mathtt{dig_{chall_1}}$ obtained by hashing the two commitments and the message digest (lines 13 – 16). This enables to compute the first responses $\mathbf{y}[i]$ (lines 17–19), whose digest $\mathtt{dig_{chall_2}}$ serves as seed for the generation of the second challenge (lines 20–21). The signature is finally assembled according to the bits of the second challenge string $\mathtt{chall}_2$ (lines 22–27).

CROSS.Verify. Verification (shown in Algorithm 3) first generates the public matrices $\overline{M}, H^\top$ (lines 1–2) from the public key and reconstructs the values of $\mathtt{chall}_1$ and $\mathtt{chall}_2$ from the message and material contained in the signature (lines 3–6). Using seed tree nodes contained in the signature, depending on $\mathtt{chall}_2$ the verifier recomputes a subset of round seeds via RebuildLeaves (line 7). Afterwards, the verifier reconstructs the values $\mathtt{dig_{cmt_0}}, \mathtt{dig_{cmt_1}}$, computing in turn the missing $\mathtt{cmt}_0[i]$ for the Merkle tree (for $\mathtt{dig_{cmt_0}}$) and the missing $\mathtt{cmt}_1[i]$ (for $\mathtt{dig_{cmt_1}}$) in lines 8–20. Besides that, the verifier reconstructs all the responses $\mathbf{y}[i]$ to the first challenge. This lets the verifier compute the expected values

Algorithm 3 CROSS.VERIFY

Require: $\mathsf{pk} = (\mathsf{seed}_{\mathsf{pk}} \in \{0,1\}^{2\lambda}, \mathbf{s} \in \mathbb{F}_p^{n-k})$, $\mathsf{msg} \in \{0,1\}^*$,
$\mathsf{sig} = (\mathsf{salt} \in \{0,1\}^{2\lambda}, \mathsf{dig}_{\mathsf{cmt}} \in \{0,1\}^{2\lambda}, \mathsf{dig}_{\mathsf{chall}_2} \in \{0,1\}^{2\lambda}, \mathsf{path}, \mathsf{proof}, \mathsf{rsp})$
Ensure: $\mathsf{valid} \in \{\mathsf{true}, \mathsf{false}\}$

1: $\overline{W}, V^\top \leftarrow \text{CSPRNG}(\mathsf{seed}_{\mathsf{pk}}, \mathbb{F}_z^{m\times(n-m)} \times \mathbb{F}_p^{k\times(n-k)})$ ▷ *sample matrices from* $\mathsf{seed}_{\mathsf{pk}}$
$\overline{M} \leftarrow [\overline{W}, I_m]$ $V^\top \leftarrow \text{CSPRNG}(\mathsf{seed}_{\mathsf{pk}}, \mathbb{F}_p^{k\times(n-k)})$
2: $H^\top \leftarrow [V^\top \mid I_{n-k}]$
3: $\mathsf{dig}_{\mathsf{msg}} \leftarrow \text{HASH}(\mathsf{msg})$
4: $\mathsf{dig}_{\mathsf{chall}_1} \leftarrow \text{HASH}(\mathsf{dig}_{\mathsf{msg}} \| \mathsf{dig}_{\mathsf{cmt}} \| \mathsf{salt})$ ▷ *dig_{cmt} and salt from signature*
5: $\mathsf{chall}_1 \leftarrow \text{CSPRNG}(\mathsf{dig}_{\mathsf{chall}_1}, (\mathbb{F}_p^*)^t)$ ▷ *sample $chall_1$ over multiplicative subgroup $\mathbb{F}_p^\star$*
6: $\mathsf{chall}_2 \leftarrow \text{CSPRNG}(\mathsf{dig}_{\mathsf{chall}_2}, \mathcal{B}_{t,w})$ ▷ *$\mathcal{B}_{t,w}$: the set of length t bitstrings of weight w*
7: $(\mathsf{seed}[i])_{i:\mathsf{chall}_2[i]=1} \leftarrow \text{REBUILDLEAVES}(\mathsf{path}, \mathsf{chall}_2, \mathsf{salt})$ ▷ *return leaves of seed tree where $chall_2[i] = 1$*
8: **for** $i \leftarrow 0$ **to** $t-1$ **do**
9: **if** $\mathsf{chall}_2[i] = 1$ **then** ▷ *recompute $cmt_1[i]$ and first response $\mathbf{y}[i]$*
10: $\mathsf{cmt}_1[i] \leftarrow \text{HASH}(\mathsf{seed}[i] \| \mathsf{salt})$
11: $\overline{\mathbf{e}}'_G[i], \mathbf{u}'[i] \leftarrow \text{CSPRNG}(\mathsf{seed}[i] \| \mathsf{salt}, \mathbb{F}_z^m \times \mathbb{F}_p^n)$ ▷ *sample error $\overline{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$ from round seed*
$\overline{\mathbf{e}}'[i] \leftarrow \overline{\mathbf{e}}'_G[i]\overline{M}$ $\overline{\mathbf{e}}'[i], \mathbf{u}'[i] \leftarrow \text{CSPRNG}(\mathsf{seed}[i] \| \mathsf{salt}, \mathbb{F}_z^n \times \mathbb{F}_p^n)$
12: $\mathbf{e}'[i] \leftarrow g^{\overline{\mathbf{e}}[i]}$
13: $\mathbf{y}[i] \leftarrow \mathbf{u}'[i] + \mathsf{chall}_1[i]\mathbf{e}'[i]$ ▷ *recompute first response*
14: **if** $\mathsf{chall}_2[i] = 0$ **then** ▷ *recompute $cmt_0[i]$, first response $\mathbf{y}[i]$ from signature*
15: $\mathsf{cmt}_1[i] \leftarrow \mathsf{rsp}[i]_1$
$(\mathbf{y}[i], \overline{\mathbf{v}}_G[i]) \leftarrow \mathsf{rsp}[i]_0$ $(\mathbf{y}[i], \overline{\mathbf{v}}[i]) \leftarrow \mathsf{rsp}[i]_0$ ▷ *unpack tuples*
16: Check if $\overline{\mathbf{v}}_G[i] \in \mathbb{F}_z^m$ Check if $\overline{\mathbf{v}}[i] \in \mathbb{F}_z^n$ ▷ *check entry $< z$*
$\overline{\mathbf{v}}[i] \leftarrow \overline{\mathbf{v}}_G[i]\overline{M}$
17: $\mathbf{v}[i] \leftarrow g^{\overline{\mathbf{v}}[i]}$
18: $\mathbf{y}'[i] \leftarrow \mathbf{v}[i] \odot \mathbf{y}[i]$ ▷ *$\odot$: component-wise product*
19: $\mathbf{s}'[i] \leftarrow \mathbf{y}'[i]H^\top - \mathsf{chall}_1[i]\mathbf{s}$ ▷ *recompute syndrome*
20: $\mathsf{cmt}_0[i] \leftarrow \text{HASH}(\mathbf{s}'[i] \| \overline{\mathbf{v}}_G[i] \| \mathsf{salt})$ $\mathsf{cmt}_0[i] \leftarrow \text{HASH}(\mathbf{s}'[i] \| \overline{\mathbf{v}}[i] \| \mathsf{salt})$
21: $\mathsf{dig}_{\mathsf{cmt}_0}, \mathsf{dig}_{\mathsf{cmt}_1} \leftarrow \text{RECOMPUTEROOT}(\mathsf{cmt}_0, \mathsf{proof}, \mathsf{chall}_2), \text{HASH}(\mathsf{cmt}_1[0] \| \ldots \| \mathsf{cmt}_1[t-1])$
22: $\mathsf{dig}'_{\mathsf{cmt}}, \mathsf{dig}'_{\mathsf{chall}_2} \leftarrow \text{HASH}(\mathsf{dig}_{\mathsf{cmt}_0} \| \mathsf{dig}_{\mathsf{cmt}_1}), \text{HASH}(\mathbf{y}[0] \| \ldots \| \mathbf{y}[t-1] \| \mathsf{dig}_{\mathsf{chall}_1})$
23: **if** $\mathsf{dig}_{\mathsf{cmt}} \neq \mathsf{dig}'_{\mathsf{cmt}} \vee \mathsf{dig}_{\mathsf{chall}_2} \neq \mathsf{dig}'_{\mathsf{chall}_2}$ **then**
24: **return** $\mathsf{valid} = \mathsf{false}$
25: **return** $\mathsf{valid} = \mathsf{true}$

$\mathsf{dig}'_{\mathsf{cmt}}$,$\mathsf{dig}'_{\mathsf{chall}_2}$ (lines 21–22), and compare them to the ones contained in the signature (line 23).

Parameter sets. CROSS uses two small primes p and z, and the security margin against attacks to RSDP/RSDP(G) is tuned via different lengths and dimensions for the $[n, k]$ random linear codes. The scheme further includes three different trade-offs between a short signature and fast computation for each security level, denoted by the optimization corners *fast* (f), *balanced* (b) and *small* (s). This is achieved by tuning the number of CROSS-ID repetitions t and the weight of the second challenge string w. In total, the NIST submission [BBB+25a] defines 18 parameter sets as listed in Table 1.

Table 1: CROSS round 2 parameter sets as specified in [BBB+25a].

Sec. level	Parameter set	p	z	n	k	m	t	w	Sym. primitive
1 ($\lambda = 128$)	RSDP-1-{f/b/s}	127	7	127	76	–	157/256/529	82/215/488	SHAKE128
3 ($\lambda = 192$)	RSDP-3-{f/b/s}			187	111	–	239/384/580	125/321/527	SHAKE256
5 ($\lambda = 256$)	RSDP-5-{f/b/s}			251	150	–	321/512/832	167/427/762	SHAKE256
1 ($\lambda = 128$)	RSDPG-1-f{f/b/s}	509	127	55	36	25	147/256/512	76/220/484	SHAKE128
3 ($\lambda = 192$)	RSDPG-1-f{f/b/s}			79	48	40	224/268/512	119/196/463	SHAKE256
5 ($\lambda = 256$)	RSDPG-1-f{f/b/s}			106	69	48	300/356/642	153/258/575	SHAKE256

Performance bottleneck. Due to the round-based nature of CROSS executing t repetitions of CROSS-ID, the most computational intensive part is the commitment computation

phase during signature generation (lines 4–11 in Algorithm 2). Each round requires to sample two vectors $\bar{\mathbf{e}}'[i]$,$\mathbf{u}'[i] \leftarrow \text{CSPRNG}(\texttt{seed}[i]\|\texttt{salt})$, to perform several arithmetic operations (including an expensive matrix-vector multiplication $\mathbf{s}'[i] = \mathbf{u}[i]H^\top$), as well as the generation of two digests $\texttt{cmt}_0[i]$ and $\texttt{cmt}_1[i]$. As all these operations are repeated t times, an optimized execution is crucial for the overall efficiency of the design. In the following, we will focus on explaining our optimization concepts specifically for this phase. Yet, the same optimizations are applied to other parts of the algorithms if applicable.

3 Hardware Architecture

Analyzing KeyGen, Sign and Verify shows that these algorithms share most of the operations, as highlighted in Table 2. All operations required for KeyGen are also part of Sign and Verify, and only few operations (gray background) are exclusive to either Sign or Verify. We thus decided to implement all three primitives in a single, unified design with the specific parameter set being configurable at compile time. The support for multiple optimization corners or security levels for a specific hard problem requires to have the parameters n, k (and m), t and w configurable via some registers. However, we investigate the size impact of different parameter sets by offering compile time configurations, also to have a fair comparison with the literature. A size estimate for a runtime configurable design can be derived from the largest parameter set with a certain additional configuration overhead. The RSDP and RSDP(G) variants further come with different arithmetic operations, making the support for both more costly. Nevertheless, it is unlikely that applications would employ both variants. In the following sections, we discuss the concepts and architecture of the top-level design and its submodules, and explain the scheduling of KeyGen, Sign and Verify. While we focus on the RSDP version for brevity, we note that the concepts apply likewise to the RSDP(G) variant.

3.1 Design Concepts

Throughout the design of the accelerator, we adopt several key design techniques: vectorization and unrolling, parallel sampling of objects, efficient pipelining between major submodules, and parallelized scheduling.

Vectorization and unrolling. The most computationally intensive operations are the matrix-vector multiplications ($\mathbf{u}H^\top$ and $\bar{\mathbf{e}}_G\overline{M}$) as well as the SHAKE computations, both of which can be linearly accelerated by allocating more resources. The matrix-vector multiplication can be vectorized such that one vector entry is multiplied by several elements of a matrix row in each clock cycle. Regarding the SHAKE computation, we can unroll the Keccak-f round function to compute more than one round per clock cycle. In Subsection 4.1 we evaluate the design space to find the optimal performance and area trade-offs for the vectorization and unrolling w.r.t. our target metric, i.e. the AT product.

Parallel sampling. To accelerate the generation of the two vectors $\bar{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$, we implement a sampling strategy that enables sampling 1) both vectors in parallel, as well as 2) multiple vector entries in parallel. To do so, we decouple the rejection sampling process from the generation of the pseudo-randomness that is required for rejection sampling. This concept also applies to the generation of the matrices $\overline{M}$ and $H^\top$ in the RSDP(G) versions.

Efficient pipelining. We pipeline the generation and processing of data where possible to prevent slowdowns caused by extensive memory accesses. For instance, while parts of the vectors $\bar{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$ are generated, they can be processed on-the-fly up to computing the

Table 2: Major operations in CROSS with line references to Algorithm 1, Algorithm 2 and Algorithm 3.

Operation	Keygen	Sign	Verify
CSPRNG(·)	✓	✓	✓
Hash(·)	✗	✓	✓
mul mat-vec $\mathbb{F}_z$	✓ (5)	✓ (5)	✓ (11, 16)
mul mat-vec $\mathbb{F}_p$	✓ (7)	✓ (9)	✓ (19)
mul vec-vec $\mathbb{F}_p$	✗	✓ (8, 19)	✓ (13, 18)
exponentiation	✓ (6)	✓ (7, 18)	✓ (12, 17)
sub vec-vec $\mathbb{F}_z$	✗	✓ (5, 6)	✗
sub vec-vec $\mathbb{F}_p$	✗	✗	✓ (19)
add vec-vec $\mathbb{F}_p$	✗	✓ (19)	✓ (13)
Tree operations	✗	✓	✓
Compression	✓	✓	✓
Decompression	✗	✗	✓

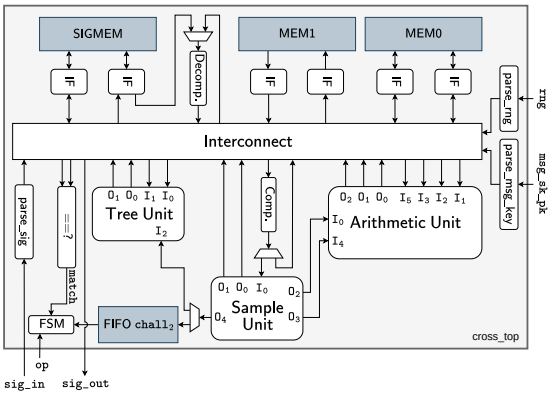


Figure 1: Block diagram of the top-level of the presented design.

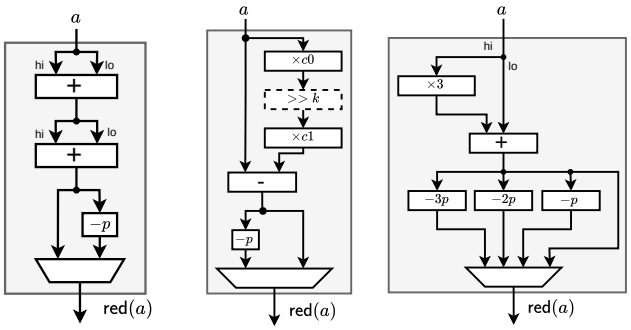
syndrome $\mathbf{s}'[i]$, without having to wait until the whole vector is generated. For objects that need to be stored for later re-use, we simultaneously store a copy of them.

Parallel scheduling. Multiple operations within CROSS are both data- and resource independent and thus, can be scheduled in parallel. For instance, during the commitment phase the computations of $\mathbf{s}'[i] = \mathbf{u}[i]H^\top$ and $\mathtt{cmt}_1[i] \leftarrow \textsc{Hash}(\mathtt{seed}[i]\|\mathtt{salt})$ can be executed in parallel without requiring additional resources, as computing $\mathbf{s}'[i]$ does not require the SHAKE module which is used to compute $\mathtt{cmt}_1[i]$. This enables to completely hide the cost of t hash computations for all $\mathtt{cmt}_1[i]$ during signature generation.

3.2 Top-level Architecture

Our design, depicted in Figure 1, contains two main computation units, the arithmetic unit and the sample unit, plus the tree unit, input parsers and the compression and decompression units. For clarity in visual representation, we omit in Figure 1 the full detail of the multiple (de-) multiplexers and point-to-point connections that enable parallel data transmission in the interconnect. All components use ARM AMBA AXI4-Stream standard interfaces and multiple field elements are transferred in 64 bit words: this width was chosen as only one padding bit is necessary for the finite field elements used in CROSS. Computations are managed by a Finite State Machine (FSM) implementing an appropriate schedule for KeyGen, Sign, and Verify.

Memory architecture. Our top-level design uses three dual-port memories, MEM0, MEM1, and SIGMEM. While MEM0 acts as the main working memory, the sequence of recomputing $\mathbf{s}'[i]$ during Verify requires access to three vectors from memory in our design. Although an additional single-port memory would resolve the port limitation of MEM0, we nevertheless instantiate MEM1 as a dual-port memory to improve pipelining: MEM1 is used only for storing $\mathbf{y}[i]$ and $\mathbf{u}'[i]$ (in place), but having two ports lets us perform the first response computation $\mathbf{y}[i] = \mathbf{u}'[i] + \mathtt{chall}_1[i]\mathbf{e}'[i]$ during Sign with simple consecutive memory accesses on each port without having to switch between read and write operations on a single port. Reading $\mathbf{u}'[i]$ on one port and writing $\mathbf{y}[i]$ on the other port thus enables efficient pipelining and prevents latencies caused by sequential read/write operations. The third memory, SIGMEM is only used to buffer the signature during Sign or Verify.



(a) Mersenne **(b)** Barrett **(c)** Crandall

Figure 2: Reduction units for 7, 127 a), 509 b) and c).

Table 3: Resource consumption for different reduction circuits.

	Mod	LUT	FF	DSPs
Mersenne	7	15	10	0
	127	38	38	0
Barrett	509	101	99	1
Crandall		54	26	0

3.3 Arithmetic Unit

The main arithmetic operations in KeyGen, Sign and Verify involve vectors over $\mathbb{F}_p$ and $\mathbb{F}_z$. In particular, CROSS uses addition, subtraction, component-wise multiplication among vectors and matrix-vector multiplication, as well as exponentiation of the generator element $g \in \mathbb{F}_p^\star$ by exponents in $\mathbb{F}_z^n$ to obtain a vector in $\mathbb{F}_p^n$.

Modular reduction. Every operation in CROSS is performed modulo one of the small prime numbers $p \in \{127, 509\}$ or $z \in \{7, 127\}$, which are all of the form $2^d - c$. The Mersenne primes $7 = 2^3 - 1$ and $127 = 2^7 - 1$ let us employ the well known Mersenne prime reduction technique that performs shifts and additions with a final conditional subtraction. Figure 2a shows our implementation of this approach: the upper d bits (hi) of a $2d$ bit multiplication result are added to the lower d bits (lo) twice, followed by a final conditional subtraction.

For $p = 509$, the software reference implementation in the NIST submission employs a generic Barrett reduction [Bar87]. This includes two multiplications (up to 27 bit operands) and two subtractions (18 bit operands), typically requiring DSP usage for efficiency. Such an implementation is depicted in Figure 2b using the precomputed constants k, $c0$ and $c1$ (note that shifting by a constant is only re-wiring in hardware). However, we can use the structure of $p = 509 = 2^9 - 3$ and implement an approach proposed by Crandall [E93] that efficiently works for primes of the form $p = 2^d - c$ with small c. The basic concept is that given $2^d \equiv c \mod p$, we can reduce an integer a by expressing it as $a = a_1 2^d + a_0$, where $a_1 = \lfloor a/2^d \rfloor$ (hi part) and $a_0 = a \mod 2^d$ (lo part), and consequently, by replacing 2^d with its congruence, $a \equiv a_1 \cdot c + a_0 \mod p$. Thus, we can implement a shift-and-add like reduction circuit as shown in Figure 2c, with parallel conditional subtractions[2]. This completely avoids multiplications and thus, does not require DSPs[3]. Table 3 lists the resource consumption of the single reduction units underlining the efficiency of the chosen Crandall reduction compared to the Barrett reduction employed in the reference software.

For the exponentiation $g^a \bmod p$ for an $a \in \mathbb{F}_z$, we make use of the small cardinality of $\mathbb{F}_z$ and implement a look-up table (as g is constant) with z entries of $\mathbb{F}_p$ elements. Such a look-up table can be implemented efficiently both on ASIC as well as FPGA: For RSDP parameters ($g = 2$) the 49 bits table is implemented in just 7 LUT3 units, and for RSDP(G) parameters ($g = 16$) the ≈ 1 KiB table takes 18 LUT6 and 9 MUX7 units.

[2]The textbook version of the reduction algorithm performs the reduction of $a \equiv 3a_1 + a_0 \mod p$ twice and performs a single conditional subtraction. However, this would require another register stage to keep the path delay short. Our experiments have shown that performing a single reduction with multiple, parallel conditional subtractions is more efficient w.r.t. resource usage, saving the additional register stage.

[3]We also considered a reduction technique presented by Solinas [Sol99] exploiting the structure $2^9 - 2^1 - 1$.

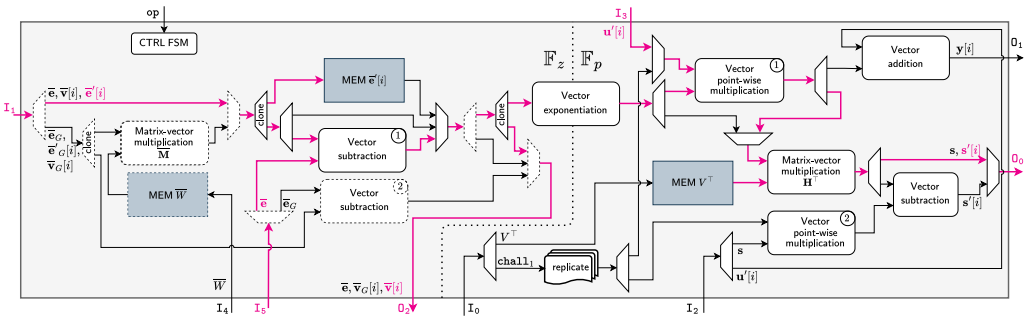


Figure 3: Arithmetic unit supporting all operations of KeyGen, Sign, and Verify. The dashed units are only required by RSDP(G) parameters. Colored paths depict the data flow within the commitment phase during Sign for the RSDP version.

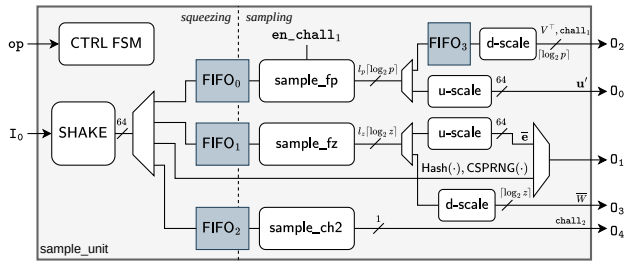
Matrix and vector arithmetic. Given the small size of the operands in CROSS, we parallelize modular addition/subtraction, exponentiation and component-wise multiplication by duplicating the corresponding compute units. Hence, all operands in a single word of the data streams are processed at once.

During the matrix-vector multiplications, only two matrices $\overline{M}$ (for RSDP(G)) and $H^\top$ are used. For the operation $\overline{\mathbf{e}} = \overline{\mathbf{e}}_G\overline{M}$, we can leverage the systematic form of $\overline{M}$ to simplify the computation $\overline{\mathbf{e}}_G\left[\overline{W}, I_m\right] = \left[\overline{\mathbf{e}}_G\overline{W}, \overline{\mathbf{e}}_G\right]$, where a smaller matrix-vector multiplication is performed to compute the first $n-m$ elements of $\overline{\mathbf{e}}$ with $m(n-m) < mn$ modular multiplications in $\mathbb{F}_z$, while the last m elements correspond to the ones of $\overline{\mathbf{e}}_G$. Likewise, the operation $\mathbf{s} = \mathbf{e}H^\top$ can leverage the systematic form of $H^\top$ computing $\mathbf{e}[V, I_{n-k}]^\top = [e_0, \ldots, e_{k-1}]\,V^\top + [e_k, \ldots, e_n]$. Thus, an accumulator is implemented that stores the result of the smaller matrix-vector multiplication of size $k(n-k)$ (instead of $n(n-k)$) in $\mathbb{F}_p$ and performs modular addition with the last $n-k$ elements of $\mathbf{e}$.

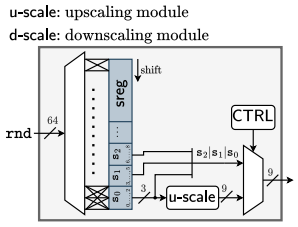
Architecture. Analyzing the arithmetic operations within KeyGen, Sign and Verify, it can be noticed that the sequence of operations is mostly fixed. We thus developed an arithmetic unit encompassing the required arithmetic modules with a semi-flexible chain structure as depicted in Figure 3. Local memories inside the arithmetic unit store the matrices $V^\top$ and $\overline{W}$, which are loaded in parallel during an initialization phase. Another local memory buffers the t error vectors $\overline{\mathbf{e}}'[i]$ obtained during the computation of the commitments $\mathtt{cmt}_0[i]$. These $\overline{\mathbf{e}}'[i]$ are re-used later to compute the first responses $\mathbf{y}[i]$ during signature generation and buffering them avoids regenerating $\overline{\mathbf{e}}'[i]$, including the costly matrix-vector multiplication $\overline{\mathbf{e}}'_G[i]\overline{M}$ in RSDP(G).

The architecture in Figure 3 is partitioned into two isolated sections separated by dashed lines. The first section involves arithmetic in $\mathbb{F}_z$ where the vector elements represent the exponents of the generator g, while the other section involves arithmetic in $\mathbb{F}_p$. In the first section, several modules (denoted by dashed borders) are RSDP(G) specific and not present in the RSDP versions. The migration between both sections happens through the vector exponentiation module that contains a small FIFO to facilitate the mapping between the number of elements in $\mathbb{F}_z$ and $\mathbb{F}_p$ per 64 bit word. Copies of modules (denoted with e.g. ②) are employed to resolve resource dependencies of data-independent operations increasing performance at little area cost. For instance, the point-wise vector-vector multiplier is instantiated twice with the first instance computing $\mathbf{y}'[i] = \mathbf{v}[i] \odot \mathbf{y}[i]$ and the second instance computing $\mathtt{chall}_1[i]\mathbf{s}$. Both multiplication results are required for the syndrome computation $\mathbf{s}'[i] = \mathbf{y}'[i]H^\top - \mathtt{chall}_1[i]\mathbf{s}$ during verification. Employing two

However, this required ≈ 30 additions/subtractions and thus, we chose the Crandall reduction approach.



(a) Structure of the sample unit.



(b) Generic sampler with parallelization $l = 3$ and 3 bit samples.

Figure 4: Block diagrams of sampling modules.

instances enables computing $\mathbf{s}'[i]$ in a pipelined manner without delay, which would not be possible when performing the two vector multiplications sequentially.

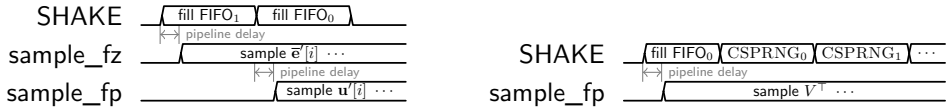
Exemplary dataflow for the commitment phase. Figure 3 depicts the dataflow of the arithmetic computations within the commitment phase for the RSDP version in color. Given the pipelining, the vectors $\overline{\mathbf{e}}'[i]$ (I_1) and $\mathbf{u}'[i]$ (I_3) are directly received from the sample unit while the secret error $\overline{\mathbf{e}}$ (I_5) is read from memory. Both $\overline{\mathbf{v}}[i]$ (O_2) and $\mathbf{s}'[i]$ (O_0) are then computed in a single flow without requiring sequential memory accesses. The dataflow in the RSDP(G) versions differs only slightly, requiring the additional matrix-vector multiplication and vector-vector subtraction in the $\mathbb{F}_z$ domain.

3.4 Sample Unit

The sample unit generates vectors and matrices with entries in $\mathbb{F}_z$ and $\mathbb{F}_p$ as well as the challenges $\texttt{chall}_1$ and $\texttt{chall}_2$. It therefore includes the SHAKE module for hashing and generation of pseudo-random data. Figure 4 shows the full sample unit (Figure 4a) as well as an exemplary rejection sampling module (Figure 4b).

SHAKE. We employ a high performance implementation of Keccak [BDPA11], where the selection between the corresponding SHAKE128 or SHAKE256 functions is configurable at synthesis time. The 1600 bit Keccak state is implemented entirely in Flip-Flops (FFs), with an additional I/O buffer (shift-register) to enable concurrent read/write operations and actual block processing. To improve the performance of the SHAKE function, we employ two measures: First, we consider unrolling x Keccak-f rounds to complete the whole operation in $^{24}/_{x}$ clock cycles at the cost of extra logic. We perform a design space exploration in Subsection 4.1 to evaluate the degree of unrolling that maximizes the AT product. Second, we enhanced the padding module such that for specific input sizes ($3\lambda + 16$ bits and $4\lambda + 16$ bits), the I/O shift-register is fast-forwarded with correctly padded input to immediately start the permutation in the following clock cycle. As these input sizes apply during seed tree-, $\texttt{cmt}_1[i]$-, as well as Merkle tree computations, this mechanism provides an efficient way of speeding up the performance given the huge amount of hash and CSPRNG operations.

Parallel sampling of objects. The sample unit is split into a *squeezing* and *sampling* part which are separated by FIFOs. This decouples the generation of pseudo-random data for the rejection samplers from the actual rejection sampling procedure, enabling *parallel sampling* of different vectors. As depicted in the timing diagram in Figure 5a, the sampling of objects in $\mathbb{F}_z$ (via sample_fz) and $\mathbb{F}_p$ (via sample_fp) can be parallelized as soon as the corresponding FIFOs contain pseudo-random data. Likewise, the SHAKE module can be



(a) Parallel sampling of $\bar{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$. **(b)** Parallel sampling of $V^\top$ and CSPRNG calls.

Figure 5: Conceptual depiction of parallel sampling of multiple objects.

used for hashing or seed expansion whenever the sampling of elements in $\mathbb{F}_z$ is inactive. Figure 5b exemplifies this for the sampling of $V^\top$ and seed tree computations.

Parallel sampling of vector entries. Besides parallel sampling on vector or matrix level, we implement rejection samplers that enable sampling multiple entries of a single vector in parallel. In the following, we denote the degree of parallelization as l_p for sampling over $\mathbb{F}_p$, and l_z for sampling over $\mathbb{F}_z$. A straightforward way to sample multiple entries in parallel is checking l_p (resp. l_z) non-overlapping chunks of the input randomness and only keeping those whose value is in the desired range. This would, however, require to sort the output samples as well as the pseudorandom input for realignment and correct output order to be compliant with the specification. For large l_p or l_z, this becomes costly in terms of wiring and multiplexing. Therefore, we employ a hybrid strategy instead as depicted in Figure 4b (example for $l = 3$): Initially, a control FSM checks if all $l = 3$ non-overlapping chunks are valid. If so, they are forwarded to the output and the shift-register (sreg) storing the input randomness (`rnd`) is shifted by $l = 3$ samples. If at least one sample needs to be rejected, the FSM switches into a sequential mode in which only a single sample is checked per clock cycle and an upscaling module (u-scale) latches only the valid ones until $l = 3$ valid samples are obtained. Afterwards, the FSM goes into its initial, parallel state again. This inherently leads to properly sorted output vectors without requiring costly "dynamic" sorting at the cost of increased latency when entering the sequential mode. Due to switching between parallel and sequential mode, selecting the proper parallelization factors l_p and l_z is crucial for overall performance. While larger l_p and l_z yield better performance if no rejections occur, the penalty for shifting out rejected entries increases likewise. Given that CROSS employs two pairs of moduli, i.e. $(z, p) \in (7, 127)$ for RSDP and $(z, p) \in (127, 509)$ for RSDP(G), and 127 and 509 are close to a power-of-two, the rejection probability for these samples is small, i.e. $1/128 \approx 0.78\%$ and $3/512 \approx 0.59\%$. However, the rejection probability for $z = 7$ is $1/8 = 12.5\%$, resulting in frequent rejections which implies switching to sequential mode. In Subsection 4.1, we explain our choice for parallelization degrees l_p and l_z on the basis of a simulated DSE.

Besides samplers for vectors over $\mathbb{F}_p$ and $\mathbb{F}_z$, sample_ch2 implements a Fisher-Yates shuffling algorithm [FY48] to generate a string of fix-weight w for $\texttt{chall}_2$. The fix-weight string itself is stored in LUTRAM given that each entry is only a single bit. In fact, a LUTRAM on a 7-series AMD FPGA can store 64×1 bits and thus, even for the CROSS instance with the largest challenge size $t = 832$, only a few LUTs are required to store it, avoiding usage of dedicated BRAMs.

3.5 Tree Unit and (De-) Compression

The remaining modules in Figure 1 are the tree unit and the modules for (de-) compression. The tree unit mainly consists of two memory controllers and corresponding memories to store the seed- and Merkle trees. These modules fetch and store the corresponding seeds and hashes and provide them to the processing modules to hide memory access time. Upon signature composition the tree unit fetches the elements that are packed into the signature and applies proper padding. Likewise, it verifies padding during verification.

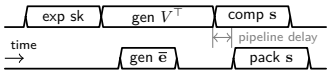


Figure 6: Scheduling of KeyGen.

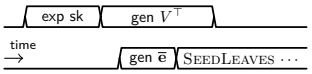


Figure 7: Scheduling of init-phase during Sign.

In general, the module is mainly responsible for data movement. The compression and decompression modules pack vectors into consecutive bitstrings or unpack them according to the CROSS specification. Given that multiple elements in $\mathbb{F}_z$ or $\mathbb{F}_p$ nicely fit into 63 bits for all parameter sets, packing or unpacking them into the 64 bit data width can be efficiently realized with shift-registers and multiplexers.

3.6 Scheduling

Generating and processing the matrices and vectors takes a substantial time in CROSS. We have optimized the scheduling in such a way that computation units are maximally loaded utilizing their parallel and pipelined architecture, and memory accesses are minimized. This is described in the following section. We note that the exact lengths of the segments in the provided timing diagrams do not model exact latencies as they differ depending over the various parameter sets. Yet, the relative positions and overlaps of segments denote sequential and parallel or pipelined scheduling.

Key generation. The RSDP KeyGen schedule is depicted in Figure 6. It starts by expanding the secret key $\mathtt{seed}_{\mathsf{sk}}$ into two additional seeds. These are used to sample the matrix $V^\top$, as well as the element $\overline{\mathbf{e}}$. Given the architecture of the sample unit, the generation of both is parallelized. The public syndrome $\mathbf{s}$ is then computed by the arithmetic unit, and compressed on-the-fly due to the pipelined architecture. For RSDP(G), the matrix $\overline{M}$ is additionally sampled in parallel to $V^\top$ and the sampled vector $\overline{\mathbf{e}}_G$ is subsequently multiplied with $\overline{M}$ to obtain $\overline{\mathbf{e}}$.

Signature generation. Sign starts with an initialization phase as depicted in Figure 7 for RSDP. It is similar to Figure 6, where instead of computing the syndrome $\mathbf{s}$, the seed tree expansion starts as soon as $\overline{\mathbf{e}}$ is fully generated, letting the tree computation overlap with the generation of $V^\top$. In the *fast* versions, SeedLeaves is computed before finishing the generation of $V^\top$ and does not contribute to the overall runtime at all. The remaining schedule of Sign is depicted in Figure 8, with a single iteration of the commitment phase shown in color. The corresponding (simplified) code snippet is shown on the left for ease of understanding. The commitment phase starts by sampling the vectors $\overline{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$ in parallel, and the former is directly processed in a pipelined manner to compute $\overline{\mathbf{v}}$. After that, the SHAKE module employed to generate the pseudo-random data for the sampling is not used until $\mathtt{cmt}_0[i]$ or $\mathtt{cmt}_1[i]$ are computed.

Then the exponentiation, pointwise multiplication of two vectors and the syndrome $\mathbf{s}'[i]$ computation follow, allowing the generation of $\mathtt{cmt}_0[i]$. These operations could be accelerated by increasing the bus width and allocating even more resources and memories for their computational units. However, a less costly and more effective way of reducing the commitment phase latency consists in scheduling the generation of $\mathtt{cmt}_1[i]$ in parallel with the syndrome computation, as both are data- and resource-independent. This completely hides the latency of $\mathtt{cmt}_1[i]$ generation, as its computation is faster than the syndrome computation, saving in total the latency corresponding to t hash computations[4].

[4]Spending more resources on accelerating computation of $\mathbf{s}'[i]$ would, at some point, prevent computing $\mathtt{cmt}_1[i]$ in parallel, as it would conflict with the computation of $\mathtt{cmt}_0[i]$ (both require the SHAKE module).

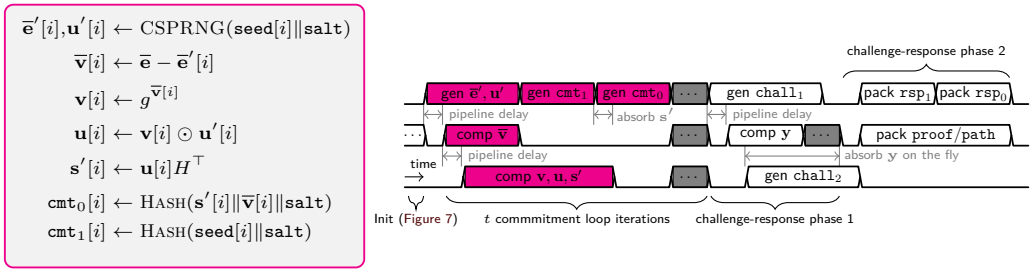


Figure 8: Scheduling of SIGN. Single commitment computation shown in color.

Once all commitments are computed, they are combined via hashing and Merkle tree computations and used to generate $\texttt{chall}_1$. As soon as the first element $\texttt{chall}_1[0]$ is available, the computation of the first responses $\mathbf{y}[i]$ is initiated, enabling to overlap the sampling of the remaining $\texttt{chall}_1$ vector with the response computation (only one element of $\texttt{chall}_1$ is required for each response $\mathbf{y}[i]$). The $\mathbf{y}[i]$ are then absorbed on-the-fly by the SHAKE module as required for computation of $\texttt{chall}_2$. This is then used to assemble the signature. In doing so, the tree unit packs the `path` and `proof` variables into the signature, while concurrently the main FSM packs the corresponding response vectors.

The scheduling of the RSDP(G) version only differs in the way $\overline{\mathbf{e}}$ is computed. In fact, it samples a shorter vector $\overline{\mathbf{e}}_G$ and multiplies it by $\overline{M}$. Also, a shorter transform $\overline{\mathbf{v}}_G$ is computed and packed in the signature.

Signature verification. Figure 9 depicts the schedule for RSDP VERIFY. After the signature is loaded and parsed accordingly, the generation of $\texttt{chall}_2$ is initiated and the generation of the public matrix is started concurrently. As soon as the pseudorandomness for both samplers is generated, $\texttt{dig}_{\texttt{chall}_1}$ can be computed by hashing the message and variables from the signature. After $\texttt{chall}_2$ has been sampled, it is used to load and decompress the proper elements from the response vectors, i.e. $\texttt{cmt}_1[i]$, $\mathbf{y}[i]$ and $\overline{\mathbf{v}}[i]$ (resp. $\overline{\mathbf{v}}_G[i]$ for RSDP(G)). Afterwards, the received `path` nodes are expanded via the REBUILDLEAVES routine of the tree unit for the *balanced* and *small* variants. For the *fast* variants, this step is not required. We then initiate the generation of $\texttt{chall}_1$, and without waiting for its completion, start the actual recomputation of the commitments. Depending on each value of $\texttt{chall}_2[i]$, an appropriate subroutine of the arithmetic unit recomputes the missing $\texttt{cmt}_0[i]$ or $\texttt{cmt}_1[i]$. Afterwards, $\texttt{dig}'_{\texttt{cmt}}$ is reconstructed and checked against the one in the signature. Likewise, $\texttt{dig}'_{\texttt{chall}_2}$ is reconstructed using the artifacts computed during VERIFY and checked against its reference in the signature.

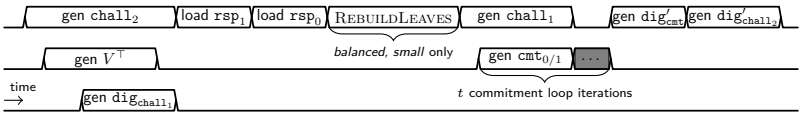


Figure 9: Scheduling of VERIFY.

4 Experimental Evaluation

We implemented our RTL design in SystemVerilog and synthesized it using Vivado 2023.1 with default settings, targeting the `xc7a200tfbg484-3` FPGA. Our designs are tested via behavioral simulations using the `cocotb` framework adopting the Verilator 5.028 simulator, and comparing the results against the C reference implementation of the latest CROSS v2.2

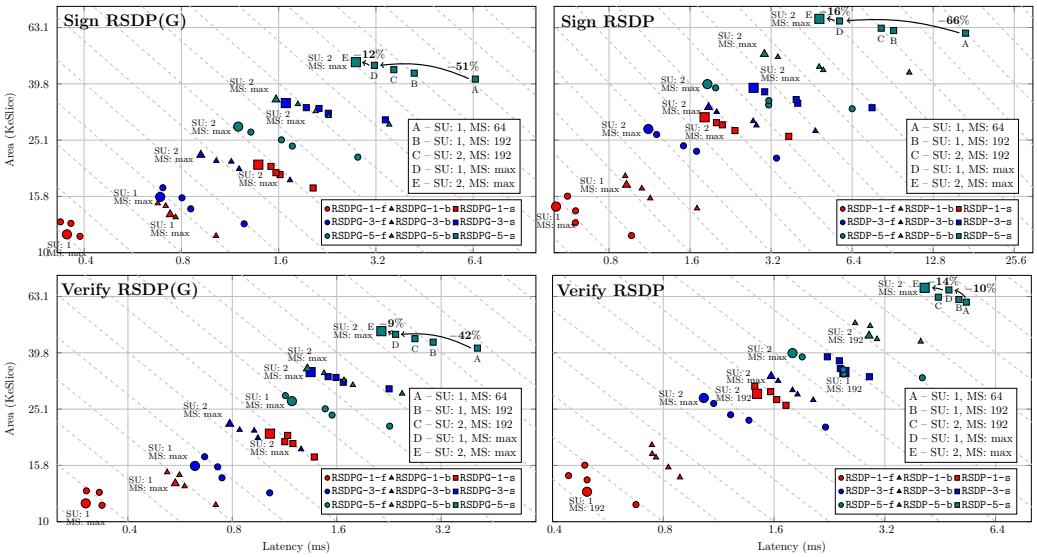


Figure 10: DSE for different configurations varying the SHAKE Unroll (SU) factor and the data width of the Matrix Stream (MS). Dashed lines denote points where the AT product is constant (closer to the origin is better).

specification. We further verified the functionality of the design by testing it on a Digilent Nexys Video board. To assess area utilization, we use the equivalent slices (eSlice) metric. It combines the usage of LUTs, FFs, BRAMs, and DSPs into a unified measure specific to the 7-series FPGA architecture by evaluating the number of LUTs and FFs obtained from the out-of-context synthesis of specialized components when the usage of BRAMs and DSPs is forbidden. Our syntheses revealed that the arithmetic operations carried out in a single DSP can be obtained with a design using 538 LUTs and 232 FFs, while a 32 kb Simple Dual Port RAM fitting in a single BRAM can be obtained from 848 LUTs and 548 FFs. Considering the slice composition in an AMD 7-Series FPGA, we compute the total number of eSlices as $\mathbf{max}(\lceil \#\text{LUTs}/4 \rceil, \lceil \#\text{FFs}/8 \rceil)$. The eSlice metric is useful for comparing FPGA designs but may overestimate area when BRAMs and DSPs are underutilized. It can also underestimate area for more complex memory designs, and does not account for frequency penalties from increased logic and routing delays when DSPs and BRAMs are excluded. By multiplying the eSlice metric by the latency (expressed in μs or ms), we derive an Area-Time (AT) efficiency metric, where more efficient designs are characterized by lower AT products.

We will now provide a design space exploration for various configurations of the arithmetic unit and sample unit, and discuss the results for the final top-level design. Finally, we compare our results with implementations of other post-quantum signature schemes[5].

4.1 Design Space Exploration and Evaluation

Keccak and matrix multiplication. Figure 10 depicts the DSE of the top-level design for each parameter set when varying the number of KECCAK-f permutation rounds performed in a single clock cycle (1 or 2) and the width of the AXI stream transferring parts of the matrix rows (192 bits, or $(n-k)\lceil \log_2 p \rceil$ bits, i.e. an entire matrix row) in the arithmetic unit. For reference, we include a compact design using 64 bit streams and performing a single KECCAK-f round per clock cycle. These design points are labeled as A to E for the

[5]For brevity, we restrict most of our discussion of results to security level 1 parameters. For all security levels, more detailed numbers and additional comparisons, we refer to the Appendix.

example of security level 5 parameters of the small variants. For an initial investigation, the parallelization degrees l_z and l_p for the rejection samplers were both set to $l_z = l_p = 1$, i.e. constituting sequential sampling, as in our design, the matrix multiplier processes at most one vector entry per clock cycle. For each parameter set, the design point with best efficiency have a bigger marker with associated configuration.

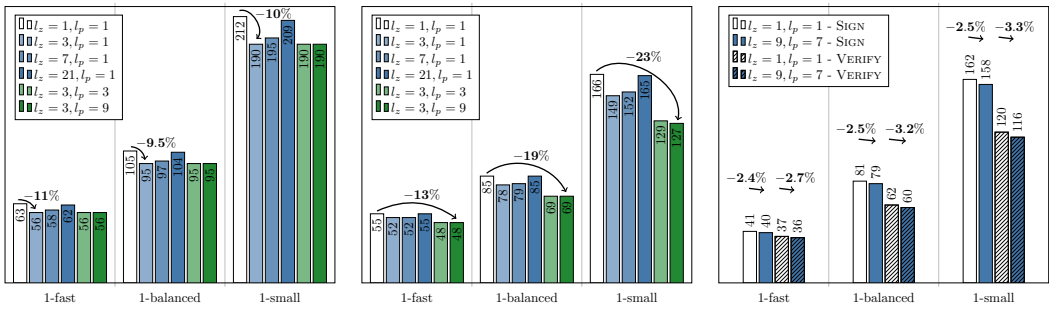
First, we do not unroll the Keccak-f round function but compare the effect of widening the matrix stream from 64 bits to the width of an entire matrix row. In the labeled parameter set, this refers to the transition from point A to D and improves performance by up to 66% for RSDP signature generation and 51% for RSDP(G). For verification, the improvements in the *small* variant of RSDP is substantially smaller (10%). This is because matrix multiplication only happens if the second challenge is zero, which is only the case in $t - w$ rounds. Yet, if the distance between t and w increases (going from *small* toward *fast* variants), it shows an increasing spread over the latency axis within one parameter set, underlining the benefit of a wider matrix stream also for RSDP verification. In the RSDP(G) verification, CROSS performs matrix multiplications (with $\overline{M}$) for both values of the second challenge, hence benefiting from the wider matrix stream in general (42%). Averaging the results over all parameter sets for Sign and Verify, this widening leads to 39.7% latency and 29.2% AT improvements at only 17.5% increased area cost.

Assuming the fully widened matrix stream, we observed that further performing two Keccak-f permutation rounds per clock cycle instead of one (transition from point D to E) offers a marginal yet beneficial improvement, up to 16% for `RSDP-5-small`. Averaging the results for Sign and Verify over all parameter sets, this unrolling results in a 5.27% increased eSlice count and frequency drop of about 5.83%, but still yields a 6.47% latency and 1.43% AT improvement.

Given the observations, we selected the configuration which on average has the highest efficiency and lowest latency, i.e. the one performing two Keccak-f permutation rounds and processing an entire matrix row in each clock cycle. Yet, we note that the configurability of our design provides a certain degree of freedom to trade performance and area consumption, enabling to adjust for application specific requirements.

Choosing sampling degrees l_z and l_p. In Subsection 3.4, we presented our architecture to sample l_z or l_p elements in $\mathbb{F}_z$ resp. $\mathbb{F}_p$ in parallel. With $(z, p) = (7, 127)$ for RSDP, and $(z, p) = (127, 509)$ for RSDP(G), 21 elements of $\mathbb{F}_7$, 9 elements of $\mathbb{F}_{127}$ and 7 elements of $\mathbb{F}_{509}$ fit into a 64 bit data word considering one padding bit. We thus evaluate parallelization degrees $l_7 \in \{1, 3, 7, 21\}$, $l_{127} \in \{1, 3, 9\}$ and $l_{509} \in \{1, 7\}$, where a value of 1 denotes the sequential case as a baseline reference. We assume the configuration selected in the previous DSE for the SHAKE unroll factor and matrix stream width.

A comparison of cycle counts for Sign and Verify is shown in Figure 11 for security level 1 parameters and different l_z and l_p. Figure 11a and Figure 11b depict the case for RSDP Sign and Verify and leads to the following conclusions: Due to the high rejection probability of $1/8 = 12.5\%$ for $z = 7$, sampling more than 3 elements in parallel has negative impact on performance as the rejection sampler frequently switches into sequential mode. In fact, for $l_z = 21$, the design is only slightly faster than the purely sequential version. Thus, a value of $l_z = 3$ yields the best performance. It is noteworthy however, that increasing l_p does not further benefit the performance of signature generation notably as shown in Figure 11a by the green bars. The reason is that the vector $\mathbf{u}[i]$ (computed from the sampled $\mathbf{u}'[i] \in \mathbb{F}_p^n$) is subsequently multiplied by the matrix H to compute $\mathbf{s}'[i] = \mathbf{u}H^\top$, where only one vector entry is processed per clock cycle. Thus, sampling multiple entries at once has no benefit. Yet, increasing l_p has a positive effect in signature verification as can be seen in Figure 11b. Unlike the previous case, $\mathbf{u}'[i]$ is used in a pointwise addition computing $\mathbf{y}[i] = \mathbf{u}'[i] + \mathtt{chall}_1[i]\mathbf{e}'[i]$, which is vectorized to full word width. Thus, increasing l_p to $l_p = 3$ or $l_p = 9$ benefits performance. While the



(a) RSDP Sign **(b)** RSDP Verify **(c)** RSDP(G) Sign and Verify

Figure 11: Cycle count ($\times 10^3$) comparison for different l_z and l_p during rejection sampling.

Table 4: Sampling performance in clock cycles averaged over 1k simulated runs. X, Y denotes parallel sampling of X and Y, $a/b/c$ denotes *fast*/*balanced*/*small* variants.

Param.	$\mathbb{F}_z^n$	$\mathbb{F}_z^n, \mathbb{F}_p^n$	$V^\top$	N_{tree}	$\texttt{chall}_2$	**Param.**	$\mathbb{F}_z^m$	$\mathbb{F}_z^m, \mathbb{F}_p^n$	$W, V^\top$	N_{tree}	$\texttt{chall}_2$
RSDP-1	134	134	3931	62	857/1379/2805	RSDPG-1	77	112	847	10	801/1378/2757
RSDP-3	168	169	8487	135	1293/2098/3158	RSDPG-3	88	135	1810	22	1216/1450/2756
RSDP-5	209	209	15201	234	1754/2757/4533	RSDPG-5	96	164	3088	37	1636/1947/3508

difference between $l_p = 3$ and $l_p = 9$ is rather small, our analysis showed that the area and frequency impact is – in relative terms – even smaller, leading to a slightly (though minimal) better AT product. As a result, we chose the configuration $(l_z, l_p) = (3, 9)$ for the RSDP case.

For the RSDP(G) parameters shown in Figure 11c, the situation is different. In both Sign and Verify, every sampled vector is multiplied by a matrix (either $\overline{M}$ or H), given that an exponent vector $\overline{\mathbf{e}}$ is a code obtained as $\overline{\mathbf{e}} = \overline{\mathbf{e}}_G \overline{M}$. As before, matrix multiplication negates the benefit of faster sampling, except for a minimal benefit of filling the pipeline between the sample unit and arithmetic unit faster. While this enables starting the multiplication earlier, measurements have shown that the area cost and associated routing complexity even decreases the overall latency. Therefore, parallel sampling does not improve but even worsens the efficiency for the RSDP(G) versions. We thus keep the purely sequential configuration of $(l_z, l_p) = (1, 1)$ for RSDP(G) parameter sets.

Parallel sampling of objects. To evaluate the effect of sampling multiple vectors in parallel, we computed the average cycle counts over 1k simulated runs with the previously evaluated values for l_z and l_p. The results reported in Table 4 show substantial improvements when sampling the vectors $\overline{\mathbf{e}}'[i]$ and $\mathbf{u}'[i]$ in parallel. For RSDP, $\mathbf{u}'[i]$ can be fully sampled parallel to $\overline{\mathbf{e}}'[i]$ due to the higher rejection probability of the later, which is given that the numbers in column 2 and 3 are equal. For RSDP(G) the savings are smaller as the vector $\mathbf{u}'[i]$ cannot be fully sampled in parallel to $\overline{\mathbf{e}}'_G[i]$ due to the overall shorter vector lengths and low rejection probabilities. We also benchmarked the number of nodes in the seed tree that can be expanded while initially sampling the public matrix $V^\top$, denoted as N_{tree}. For RSDP, a decent amount of seeds can be expanded, while for RSDP(G), N_{tree} is rather small since the public matrices are sampled much faster. However, due to the different structure of the seed tree in the *fast* parameter sets, this metric does not apply. Sampling $\texttt{chall}_2$ is rather slow compared to the other vectors, but its overall effect can be neglected since it is only sampled once. For $\texttt{chall}_1$ we do not state numbers explicitly since it is sampled parallel to the computation of the first round responses $\mathbf{y}[i]$.

Table 5: Synthesis results for the **Top-level** with timings for KeyGen, Sign, Verify.

Sec. level	Param.	Resources LUT	FF	BRAM	DSP	KeSlice	Freq. MHz	KeyGen ms	KeyGen AT	Sign ms	Sign AT	Verify ms	Verify AT
1	RSDP-1-f	26741	11678	45.0	0	16.2	118	0.035	0.568	0.478	7.76	0.409	6.64
	RSDP-1-b	27308	11820	57.5	0	19.0	111	0.037	0.708	0.856	16.3	0.618	11.8
	RSDP-1-s	27779	11990	111.5	0	30.6	114	0.036	1.11	1.67	51.0	1.12	34.2
	RSDPG-1-f	27324	11726	28.5	0	12.9	122	0.008	0.109	0.338	4.35	0.303	3.91
	RSDPG-1-b	28572	11892	37.0	0	15.0	120	0.009	0.129	0.676	10.1	0.519	7.78
	RSDPG-1-s	28777	12257	63.0	0	20.6	117	0.009	0.182	1.38	28.4	1.03	21.1
3	RSDP-3-f	29096	13761	97.0	0	27.8	108	0.081	2.26	1.09	30.3	0.973	27.1
	RSDP-3-b	29790	14028	121.5	0	33.2	113	0.078	2.57	1.69	56.0	1.24	41.1
	RSDP-3-s	29742	14067	148.0	0	38.8	109	0.080	3.12	2.61	101	1.80	69.7
	RSDPG-3-f	31185	13272	43.5	0	17.0	112	0.017	0.297	0.701	11.9	0.665	11.3
	RSDPG-3-b	31323	13479	68.0	0	22.2	115	0.017	0.378	0.919	20.4	0.786	17.5
	RSDPG-3-s	32123	13808	122.5	0	34.0	118	0.017	0.564	1.63	57.2	1.35	45.7
5	RSDP-5-f	32085	15537	151.0	0	40.0	113	0.138	5.51	1.71	68.4	1.60	64.1
	RSDP-5-b	33056	15635	202.0	0	51.1	111	0.140	7.16	2.73	140	2.09	107
	RSDP-5-s	34868	15815	280.0	0	68.1	105	0.148	10.1	4.62	314	3.27	223
	RSDPG-5-f	34626	15331	91.5	0	28.1	113	0.028	0.798	1.20	33.6	1.14	31.9
	RSDPG-5-b	36033	15502	122.5	0	35.0	113	0.028	0.995	1.57	54.9	1.32	46.1
	RSDPG-5-s	35296	15853	182.5	0	47.5	113	0.028	1.35	2.77	132	2.15	102

Table 6: Arithmetic Unit resources.

Param.	LUT	FF	BRAM	KeSlice
RSDP-1-{f/b/s}	8089	5815	15/15/19	5.20/5.20/6.05
RSDP-3-{f/b/s}	10809	7768	24/24/34.5	7.79/7.79/10.0
RSDP-5-{f/b/s}	13660	9756	28/38.5/52.5	9.35/11.5/14.5
RSDPG-1-{f/b/s}	9901	6039	11/11/15	4.81/4.81/5.65
RSDPG-3-{f/b/s}	14248	7919	12/16/26.5	6.11/6.95/9.18
RSDPG-5-{f/b/s}	17881	9887	20/30.5/30.5	8.71/10.9/10.9

Table 7: Sample Unit resources.

Param.	LUT	FF	BRAM	KeSlice
RSDP-1-f	14732	4049	4.0	4.53
RSDP-1-b	14643	4048	4.0	4.51
RSDP-1-s	14364	4072	4.0	4.44
RSDPG-1-f	13647	3887	3.5	4.15
RSDPG-1-b	13833	3886	3.5	4.20
RSDPG-1-s	13898	3902	3.5	4.22

4.2 Final Design

The synthesis results of our final design for the Artix-7 FPGA with timings averaged over 10k runs is reported in Table 5. In the benchmarks we chose a message size of 59 bytes as done by benchmarking frameworks like SUPERCOP [BL] and pqm4 [KPR+].

Area consumption. When comparing the RSDP(G) based designs with the RSDP ones, the number of required BRAMs drops on average by 38.4%, while requiring only 5.43% more LUTs, resulting in a 27.8% improvement in the eSlice area metric, in part due to a 4.14% faster clock frequency. Like other code-based signature schemes, CROSS requires a considerable amount of BRAM, which limits the compatibility of this top-level design with some products of the AMD Artix-7 family. The top-of-the-line `xc7a200t` chip can fit every parameter set, and only 5 out of 18 designs do not fit in the mid-range `xc7a100t` chip.

Table 6 reports the synthesis results of the arithmetic unit for the different parameter sets. Roughly 2/3 of its area are due to the matrix-vector multipliers, while most of the remaining logic is caused by the component-wise vector multipliers. The local memories storing the matrices $V^\top$ and $\overline{W}$ and the vectors $\overline{\mathbf{e}}'[i]$ occupy from 11 to 28 BRAM resources in the *fast* corner, and up to 52.5 BRAMs for `RSDP-5-s`. The occupation of the remaining resources in the FPGA fabric do not vary depending on the optimization corner.

The resource consumption of the sample unit is shown in Table 7. The logic cost varies only slightly between *fast*, *balanced* and *small* corners, which is mainly due to the varying lengths of $\texttt{chall}_1$ and $\texttt{chall}_2$ (having t entries). It is worth considering that the SHAKE core within the sample unit takes on average $\approx 34\%$ of the LUTs and $\approx 20\%$ FFs used by the whole top-level design (due to unrolling two permutation rounds). Inspecting the

numbers for the arithmetic unit and sample unit and the total resource consumption shows that those modules are the main contributors and the remaining sub-modules and control logic play a relatively minor role in the area discussion.

Performance evaluation. The KeyGen operation takes from 8 to 148 μs depending on the parameter set, while Sign instead ranges from 338 μs to 4.62 ms and Verify takes from 303 μs to 3.27 ms. The RSDP(G) parameters show an average latency improvement of $\approx 33\%$ for signing and $\approx 27\%$ for verification with respect to RSDP, while also improving the AT efficiency metric on average by 51% and 47%.

We also analyzed how much time our design spends in the *online* phase, corresponding to the operations working on material derived from the message to be signed, in proportion to its *offline* phase, which consists of operations that can be performed before the message is available. The results showed that the *online* phase accounts for about 11% of the runtime for the *small* and *balanced* parameter sets, and about 14% for the *fast* parameter sets. Likewise, about $85 - 90\%$ of the signature generation algorithm can be precomputed in advance without waiting for the message to be signed. This can be hugely beneficial in specific applications that can split the computation into offline and online phases.

4.3 Comparison with Related Work

We now compare our results with state-of-the-art hardware implementations for other signature schemes, as well as optimized software implementations of CROSS.

Comparison with hardware implementations. Figure 12 compares our CROSS design with hardware accelerators for post-quantum signatures limited to the security level 1 parameters (resp. 2 for CRYSTALS-Dilithium) and designs available for the Artix-7 FPGA. It shows the latency, area, and efficiency of the Sign and Verify primitives. However, we note that comparison with these works must be taken with care since already the schemes themselves are inherently different. Our design represents a compelling alternative to CRYSTALS-Dilithium, Falcon and SPHINCS$^+$ (blue points). In particular, signature generation has similar efficiency figures to some CRYSTALS-Dilithium and SPHINCS$^+$ counterparts, but performance does not scale as well for verification. However, when compared to other hardware implementations for the additional NIST on-ramp signatures (red points; round 1 parameters as no newer versions were available), such as the ones for LESS [BWMG23], MEDS [DLN+25], Raccoon [NKZ+25], MAYO [HSK+24] and SDitH [DHSY24], our design shows remarkable numbers[6]. Our CROSS implementation yields better results than the works for LESS, MEDS and SDitH in any metric, except for area consumption for the `RSDP-1-s` parameter set when compared with LESS due to our area footprint being dominated by the high BRAM count. The high efficiency of our designs is comparable to the ones for MAYO, while occupying less area. Compared to MAYO, performance of the CROSS verification is slightly worse, but some parameter sets show faster signing times. While our design can compete with the hardware implementation of Raccoon in terms of performance and efficiency, it has higher area cost. Thus, our implementation is generally placed between the design of the multivariate-based MAYO and the lattice-based Raccoon in terms of performance and area at roughly the same efficiency.

Comparison with high-performance software. Compared to the optimized AVX2 software implementation from the CROSS specification [BBB+25a, Table 8], it shows that our hardware accelerator reduces the cycle count for Sign on average by 26.8×/19.9× (R-SDP/RSDP(G)), and by 23.3×/16.4× (RSDP/RSDP(G)) for Verify. For KeyGen, we

[6] Another implementation of MAYO has been presented in [SMA+24] but we omit it from the comparison since it uses obsolete parameters and is less efficient than the work presented in [HSK+24].

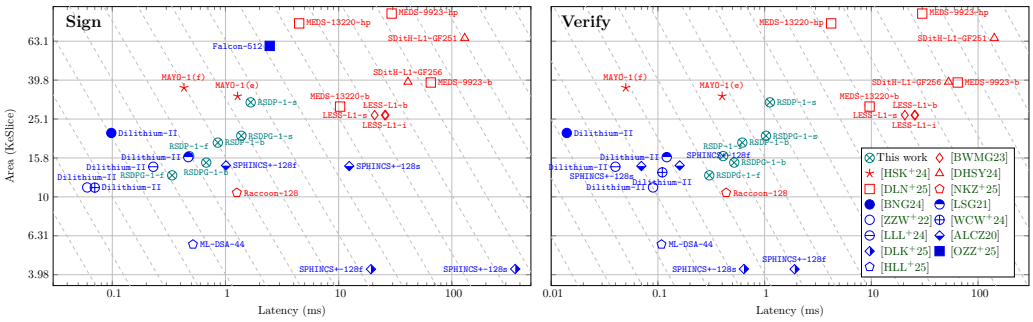


Figure 12: Results for SIGN and VERIFY primitives for security level 1 post-quantum signatures on an Artix-7 FPGA – logarithmic scale axes, dashed lines denote points in space with equal efficiency (closer to the origin is better, every step is a 2× improvement).

achieve an average cycle count reduction of about 12.6× for RSDP and 28× for RSDP(G). In general, the improvements are slightly lower for the *fast* variant, and slightly higher for the *small* variants. While the AVX2 benchmarks run on a desktop-class CPU with substantially higher power consumption at a clock frequency of 3.6GHz, our design can run on a `xc7a200tfbg484-3` FPGA at $\approx 1/36$ of the clock frequency consuming $\approx 1/40$ of power, resulting on average in a moderate 2.56×/1.15×/1.44× (for RSDP) and 1.09×/1.55×/1.92× (for RSDP(G)) slower runtime than the AVX2 CPU for KEYGEN/SIGN/VERIFY. Given the extremely different platforms, this comparison is given for completeness and contextualizing of results rather than for efficiency and performance rankings. For completeness, we compared our performance with the C reference implementation running on an Intel i7-10700 at 2.9GHz with AVX2 and SSE extensions disabled. Results have shown that our design outperforms this software by up to 7.56× when signing a message.

Comparison with software for constrained devices. Optimized software implementations of CROSS for embedded devices have been presented in [SGB+25, HMK+24]. We compare our performance with the software implementation in [HMK+24, Table 5, 'light'] optimized for the ARM Cortex-M4 on a STM32L4R5ZI device, as it is faster than the corresponding one reported in [SGB+25]. For the RSDP versions, our design reduces the cycle count for SIGN by factors of between 302× – 430×, and between 189× – 290× for VERIFY. The performance advantage of our design tends to increase from *fast* to *small* versions and toward higher security levels. For KEYGEN, the clock cycles reduce by about 106× – 117×. For the RSDP(G) versions, the performance gains are slightly lower, showing factors of 194× – 232× for KEYGEN, 274× – 344× for SIGN, and 167× – 246× for VERIFY. As the target board in [HMK+24] can run at 120 MHz, which is close to the frequency of our hardware design, the speedup factors for the cycle counts given above can also be interpreted as rough approximations for the speedups w.r.t. absolute runtime on the Cortex-M4 platform running at full speed.

5 Conclusion

In this work, we presented the first hardware implementation of CROSS, a second round candidate in NISTs on-ramp signature call. Our design supports all 18 parameter sets yielding good overall performance at relatively low resource cost. In fact, our design is compact enough to deploy all 18 parameter sets on AMD's Artix-7 FPGAs, despite the inherently high memory consumption of signature schemes based on zero-knowledge proofs. One core optimization is the parallelized sampling of both, multiple vectors, as

well as multiple vector elements. Furthermore, proper pipelining and careful scheduling of operations enables hiding the cost of operations without the need to spend additional computing resources. By using a hardware friendly circuit for modular reduction, we obtain a DSP free design which benefits portability to platforms without such dedicated processing units. The configurability of submodules enables further trade-offs of performance and area utilization at similar efficiency, allowing to adapt the design to the needs of different use-cases. Our design is constant time and faster than most on-ramp signature schemes while being among the smallest at the same time. We consider our design as a starting point for further design space explorations that include computing several iterations of the zero-knowledge protocol in parallel, as well as integrating side-channel protection.

Acknowledgements

This work was partially supported by project SERICS (PE00000014) under the Italian MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU. The authors also acknowledge the financial support by the Federal Ministry of Research, Technology and Space of Germany in the programme of "Souverän. Digital. Vernetzt.". Joint project 6G-life, project identification number: 16KISK002

References

[ALCZ20] Dorian Amiet, Lukas Leuenberger, Andreas Curiger, and Paul Zbinden. FPGA-based SPHINCS$^+$ implementations: Mind the glitch. In *23rd Euromicro Conference on Digital System Design, DSD 2020, Kranj, Slovenia, August 26-28, 2020*, pages 229–237. IEEE, 2020.

[Bar87] Paul Barrett. Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 311–323. Springer, Berlin, Heidelberg, August 1987.

[BBB+25a] Marco Baldi, Alessandro Barenghi, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. CROSS documentation v2.2. [Online]. Available from: https://web.archive.org/web/20250809061942/https://www.cross-crypto.com/CROSS_Specification_v2.2.pdf, (Archived on 9 Aug. 2025), 2025.

[BBB+25b] Marco Baldi, Alessandro Barenghi, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. CROSS security details. [Online]. Available from: https://web.archive.org/web/20250809060242/https://www.cross-crypto.com/CROSS_SecurityDetails_v2.2.pdf, (Archived on 9 Aug. 2025), 2025.

[BDPA11] Guido Bertorni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak reference. Accessed: Oct. 2024, 2011.

[BKP20] Ward Beullens, Shuichi Katsumata, and Federico Pintore. Calamari and Falafl: Logarithmic (Linkable) Ring Signatures from Isogenies and Lattices. In Shiho

Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 464–492. Springer, 2020.

[BL] Daniel J. Bernstein and Tanja Lange. eBACS: ECRYPT benchmarking of cryptographic systems. https://bench.cr.yp.to. accessed 4 April 2025.

[BNG21] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. High-performance hardware implementation of CRYSTALS-Dilithium. In *International Conference on Field-Programmable Technology, (IC)FPT 2021, Auckland, New Zealand, December 6-10, 2021*, pages 1–10. IEEE, 2021.

[BNG24] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. Hardware accelerators for digital signature algorithms Dilithium and FALCON. *IEEE Des. Test*, 41(5):27–35, 2024.

[BWMG23] Luke Beckwith, Robert Wallace, Kamyar Mohajerani, and Kris Gaj. A high-performance hardware implementation of the LESS digital signature scheme. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023*, pages 57–90. Springer, Cham, August 2023.

[DHSY24] Sanjay Deshpande, James Howe, Jakub Szefer, and Dongze Yue. SDitH in hardware. *IACR TCHES*, 2024(2):215–251, 2024.

[DLK+25] Sanjay Deshpande, Yongseok Lee, Cansu Karakuzu, Jakub Szefer, and Yunheung Paek. SPHINCSLET: An area-efficient accelerator for the full SPHINCS+ digital signature algorithm. *ACM Trans. Embed. Comput. Syst.*, April 2025. Just Accepted.

[DLN+25] Sanjay Deshpande, Yongseok Lee, Mamuri Nawan, Kashif Nawaz, Ruben Niederhagen, Yunheung Paek, and Jakub Szefer. Unified MEDS accelerator. Cryptology ePrint Archive, Paper 2025/796, 2025.

[E93] Crandall Richard E. Method and apparatus for public key exchange in a cryptographic system, 1993. Available at: https://patents.google.com/patent/US5159632A/en.

[FHK+20] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-Fourier lattice-based compact signatures over ntru, October 2020. Submission to the NIST Post-Quantum Cryptography Standardization Process, Specification v1.2 – 01/10/2020.

[FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986.

[FY48] Ronald Aylmer Fisher and Frank Yates. *Statistical tables for biological, agricultural and medical research.* Oliver and Boyd, London, 3rd ed., rev. and enl edition, 1948.

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *J. ACM*, 33(4):792–807, 1986.

[HLL+25] Tianze Huang, Jiahao Lu, Aobo Li, Lei Chen, Kai Li, Jiaming Zhang, Mingbo Wang, Xianqi Mei, Hao Li, and Dongsheng Liu. A lightweight and efficient post quantum crypto-processor for ml-dsa. *IEEE Transactions on Circuits and Systems I: Regular Papers*, pages 1–13, 2025.

[HMK+24] Harry Hart, Puja Mondal, Suparna Kundu, Supriya Adhikary, Angshuman Karmakar, and Chaoyun Li. LightCROSS: A secure and memory optimized post-quantum digital signature CROSS. Cryptology ePrint Archive, Paper 2024/1929, 2024. https://eprint.iacr.org/2024/1929, v20251023:081431.

[HSK+24] Florian Hirner, Michael Streibl, Florian Krieger, Ahmet Can Mert, and Sujoy Sinha Roy. Whipping the multivariate-based MAYO signature scheme using hardware platforms. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 3421–3435. ACM, 2024.

[KPR+] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. https://github.com/mupq/pqm4.

[LLL+24] Xiang Li, Jiahao Lu, Dongsheng Liu, Aobo Li, Shuo Yang, and Tianze Huang. A high speed post-quantum crypto-processor for Crystals-Dilithium. *IEEE Trans. Circuits Syst. II Express Briefs*, 71(1):435–439, 2024.

[LSG21] Georg Land, Pascal Sasdrich, and Tim Güneysu. A hard crystal - implementing Dilithium on reconfigurable hardware. In Vincent Grosso and Thomas Pöppelmann, editors, *Smart Card Research and Advanced Applications - 20th International Conference, CARDIS 2021, Lübeck, Germany, November 11-12, 2021, Revised Selected Papers*, volume 13173 of *Lecture Notes in Computer Science*, pages 210–230. Springer, 2021.

[MMP+24] Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, Noah Waller, and Yi-Kai Liu. Status Report on the First Round of Additional Digital Signature Schemes for Post-Quantum Cryptography. Technical report, National Institute of Standards and Technology (U.S.), 2024.

[NKZ+25] Ziying Ni, Ayesha Khalid, Zhaoyu Zhang, Yijun Cui, Weiqiang Liu, and Máire O'Neill. HRaccoon: A high-performance configurable SCA resilient Raccoon hardware accelerator. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(3):413–436, Jun. 2025.

[oSU15] National Institute of Standards and Technology (U.S.). SHA-3 standard: Permutation-based hash and extendable-output functions. NIST standard FIPS 202, 2015. https://doi.org/10.6028/NIST.FIPS.202.

[oSU24a] National Institute of Standards and Technology (U.S.). Module-lattice-based digital signature standard. NIST standard FIPS 204, 2024. https://doi.org/10.6028/NIST.FIPS.204.

[oSU24b] National Institute of Standards and Technology (U.S.). Module-lattice-based key-encapsulation mechanism standard. NIST standard FIPS 203, 2024. https://doi.org/10.6028/NIST.FIPS.203.

[oSU24c] National Institute of Standards and Technology (U.S.). Stateless hash-based digital signature standard. NIST standard FIPS 205, 2024. https://doi.org/10.6028/NIST.FIPS.205.

[oSU25] National Institute of Standards and Technology (U.S.). Post-quantum cryptography website. [Online]. Available from: https://web.archive.org/web/20250313133945/https://csrc.nist.gov/Projects/post-quantum-cryptography/publications, (Archived on 13 Mar. 2025), 2025.

[OZZ+25] Yi Ouyang, Yihong Zhu, Wenping Zhu, Bohan Yang, Zirui Zhang, Hanning Wang, Qichao Tao, Min Zhu, Shaojun Wei, and Leibo Liu. FalconSign: An efficient and high-throughput hardware architecture for Falcon signature generation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(1):203–226, 2025.

[SGB+25] Jonas Schupp, Marco Gianvecchio, Alessandro Barenghi, Patrick Karl, Gerardo Pelosi, and Georg Sigl. Optimizing the post quantum signature scheme CROSS for resource constrained devices. Cryptology ePrint Archive, Paper 2025/1928, 2025. https://eprint.iacr.org/2025/1928.

[SMA+24] Oussama Sayari, Soundes Marzougui, Thomas Aulbach, Juliane Krämer, and Jean-Pierre Seifert. HaMAYO: A fault-tolerant reconfigurable hardware implementation of the MAYO signature scheme. In Romain Wacquez and Naofumi Homma, editors, *COSADE 2024*, volume 14595 of *LNCS*, pages 240–259. Springer, Cham, April 2024.

[Sol99] Jerome A Solinas. *Generalized Mersenne Numbers*. Faculty of Mathematics, University of Waterloo, 1999. Available at https://cacr.uwaterloo.ca/techreports/1999/corr99-39.pdf.

[STW24] Maximilian Schöffel, Hiandra Tomasi, and Norbert Wehn. HW/SW implementation of MiRitH on embedded platforms, 2024.

[WCW+24] Zixuan Wu, Rongmao Chen, Yi Wang, Qiong Wang, and Wei Peng. An efficient hardware implementation of Crystal-Dilithium on FPGA. In Tianqing Zhu and Yannan Li, editors, *Information Security and Privacy - 29th Australasian Conference, ACISP 2024, Sydney, NSW, Australia, July 15-17, 2024, Proceedings, Part II*, volume 14896 of *Lecture Notes in Computer Science*, pages 64–83. Springer, 2024.

[ZZW+22] Cankun Zhao, Neng Zhang, Hanning Wang, Bohan Yang, Wenping Zhu, Zhengdong Li, Min Zhu, Shouyi Yin, Shaojun Wei, and Leibo Liu. A compact and high-performance hardware architecture for CRYSTALS-Dilithium. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):270–295, 2022.

A Additional Submodule Description

A.1 Sample Unit

Resource results. Table 8 lists the resource consumption of the whole sample unit with 2 unrolled rounds of the KECCAK permutation for all security levels.

Table 8: Sample Unit synthesis results for all security levels.

Param.	Resources LUT	FF	BRAM	eSlice	Freq. MHz	Param.	Resources LUT	FF	BRAM	eSlice	Freq. MHz
RSDP-1-f	14732	4049	4.000	4531	121	RSDPG-1-f	13647	3887	3.500	4154	128
RSDP-1-b	14643	4048	4.000	4509	119	RSDPG-1-b	13833	3886	3.500	4201	119
RSDP-1-s	14364	4072	4.000	4439	120	RSDPG-1-s	13898	3902	3.500	4217	119
RSDP-3-f	11616	3808	5.000	3964	135	RSDPG-3-f	11766	3647	3.500	3684	126
RSDP-3-b	14143	3819	5.000	4596	110	RSDPG-3-b	11697	3662	3.500	3667	139
RSDP-3-s	14241	3831	5.000	4621	120	RSDPG-3-s	11732	3660	3.500	3675	136
RSDP-5-f	11497	3831	7.000	4359	140	RSDPG-5-f	10738	3670	3.500	3427	127
RSDP-5-b	14378	3832	7.000	5079	125	RSDPG-5-b	10975	3668	3.500	3486	142
RSDP-5-s	14420	3849	7.000	5089	124	RSDPG-5-s	10976	3680	3.500	3486	138

A.2 Compression and Decompression Units

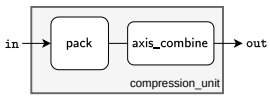


Figure 13: Compression unit.

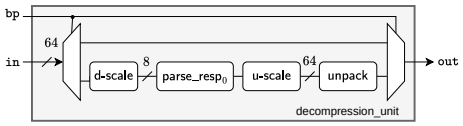


Figure 14: Decompression unit.

Compression. The compression unit shown in Figure 13 bit packs the syndrome $\mathbf{s}$ in the public key as well as the $\mathtt{rsp}_0$ elements $\mathbf{y}[i]$ and $\overline{\mathbf{v}}[i]$ (resp. $\overline{\mathbf{v}}_G[i]$ for RSDP(G)) in the signature. Likewise, it compresses $\mathbf{s}'[i]$ and $\overline{\mathbf{v}}[i]$ (resp. $\overline{\mathbf{v}}_G[i]$) to a consecutive bitstring before being absorbed by the SHAKE module when computing $\mathtt{cmt}_0[i]$. The module pack consists of a small register storing the first 63 bits received via the input `in` and concatenates them with the first bit of the next input word, while storing the remaining 62 data bits in the register. This process repeats using a counter to keep track of the amount to be shifted until a full vector is compressed. The output is zero-padded to a multiple of bytes and sent to the axis_combine module that concatenates multiple data streams. Hence, axis_combine also consists of a data register to potentially store partial words and corresponding multiplexers to output a single contiguous stream.

Decompression. The decompression unit shown in Figure 14 unpacks $\mathtt{rsp}_0$ contained in the signature. While the unpack module performs the inverse operation of the pack module, the decompression also contains a parser module for $\mathtt{rsp}_0$. Since $\mathtt{rsp}_0$ consists of tuples of $\mathbf{y}[i]$ and $\overline{\mathbf{v}}[i]$ (resp. $\overline{\mathbf{v}}_G[i]$ for RSDP(G)) and both vectors are of different lengths, they are split by the module $\mathsf{parse\_resp}_0$ before being unpacked. Otherwise, the unpacking would require additional logic since a 64 bit data word could contain bytes of both vectors $\mathbf{y}[i]$ and $\overline{\mathbf{v}}[i]$ (or $\overline{\mathbf{v}}_G[i]$). For the splitting, the input is downscaled to a single byte, separated in $\mathsf{parse\_resp}_0$ and upscaled again to 64 bit before unpacking. The signal `bp` enables bypassing the decompression for other elements in the signature.

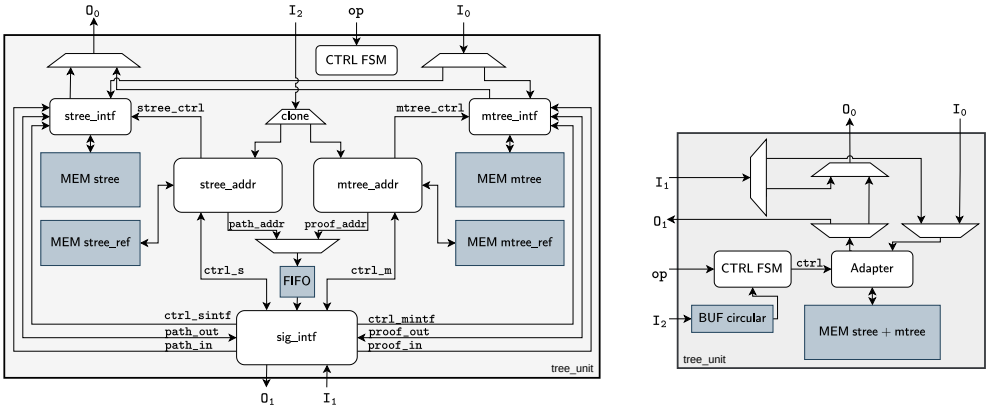
Resource results. The resource consumption of the compression and decompression modules are listed in Table 9. In general, both modules are rather compact and can operate

at relatively high frequency since they only consist of a few counters, buffering registers and multiplexers. Although the decompression module is more complex from a design perspective, it is more compact than its compression counterpart. The is due to the axis_combine module in the compression unit. Concatenating two data streams requires multiplexing every byte position of the input stream to every byte position in the output stream which does not scale well with increasing stream widths.

Table 9: Synthesis results for **Compression/Decompression Units**.

Param.	Resources			Freq.	Param.	Resources			Freq.
	LUT	FF	eSlice	MHz		LUT	FF	eSlice	MHz
RSDP-1	938/603	153/97	235/151	186/191	RSDPG-1	890/374	155/88	223/94	198/206
RSDP-3	906/521	154/105	227/131	192/176	RSDPG-3	931/503	153/94	233/126	198/192
RSDP-5	913/516	156/112	229/129	198/181	RSDPG-5	911/550	153/98	228/138	198/188

A.3 Tree Unit



(a) Small and balanced versions. **(b)** Fast version.

Figure 15: Block diagram of the tree units in CROSS.

The balanced and small versions of CROSS employ a truncated seed- and Merkle tree to compress the signature size. In the fast version, the trees have a different structure and only consist of three levels having 1, 4 and t nodes. Thus, we implement two separate modules for the corresponding parameter sets to handle the tree operations, i.e. compute addresses, store and provide tree nodes and assembling the `path` and `proof`. Both variants are depicted in Figure 15 and explained in the following.

Balanced and Small Versions. Figure 15a depicts the tree unit in the balanced and small instances. The central modules are the stree_addr and mtree_addr modules that implement the address generation logic required to traverse through the trees and set corresponding memory operations to read and write nodes from the memories that store the seed- and Merkle tree. Although the seed- and Merkle tree share the same structure, their associated operations differ slightly, such that we decided to implement two separate modules for the address generation. During tree generation (i.e. SeedLeaves and TreeRoot), the active module (stree_addr or mtree_addr) sends or receives nodes from the trees directly to or from the sample unit.

For assembling the `path` and `proof` during SeedPath and TreeProof, the controllers receive $\texttt{chall}_2$ via input $\texttt{I}_2$ and each compute a reference tree with single bit nodes to

indicate which nodes of the actual trees to pack into the signature. These reference trees are stored in separate memories implemented as LUTRAM. With these reference trees, the addresses of the nodes to include in the signature are obtained and forwarded to the signature controller sig_ctrl. This controller is directly connected to the signature memory and handles the composition of the signature in Sign by requesting nodes from the trees according to the addresses received. These nodes are are padded if required. During Verify, the controller receives the nodes from the signature and forwards them while checking for correct padding. Meanwhile, the stree_addr and mtree_addr modules compute the addresses to regenerate the required round seeds and Merkle root according to RebuildLeaves and RecomputeRoot.

Fast Version. Figure 15b shows the case for the tree unit of the fast variants. Given its simpler structure, no separate modules for address generation and tree traversals are implemented so we store both trees in a single memory. Likewise, no reference trees for the selection of the `path` and `proof` nodes is required since only a subset of exactly w leaves needs to be picked, but no inner nodes.

Resource results. The simpler structure of the fast versions is also reflected in the resource numbers given in Table 10. It yields both the highest frequencies as well as the lowest resource consumption. Also the logic in terms of LUTs and FFs is much smaller as no tree must be traversed, but all the nodes are fully linear at consecutive addresses.

The frequency is in general decreasing with the amount of BRAM required in the design. The reason for that is mostly due to the address widths and corresponding address computations. In general, however, the logic cost of the modules is rather small and the area is dominated by the memories. Since both trees in the balanced and small have $2t - 1$ nodes each, the BRAM cost is a major contributor in the overall design.

Table 10: Tree Unit synthesis results.

Param.	Resources				Freq.	Param.	Resources				Freq.
	LUT	FF	BRAM	eSlice	MHz		LUT	FF	BRAM	eSlice	MHz
RSDP-1-f	380	102	2	519	189	RSDPG-1-f	339	99	2	509	193
RSDP-1-b	975	253	6.5	1622	181	RSDPG-1-b	961	253	6.5	1619	180
RSDP-1-s	1164	298	28.5	6333	143	RSDPG-1-s	1026	268	12.5	2907	166
RSDP-3-f	424	106	8	1802	169	RSDPG-3-f	372	105	4	941	171
RSDP-3-b	1146	287	28.5	6329	133	RSDPG-3-b	1141	288	12.5	2936	143
RSDP-3-s	1207	303	28.5	6344	135	RSDPG-3-s	1102	282	28.5	6318	133
RSDP-5-f	389	111	8	1794	175	RSDPG-5-f	430	113	8	1804	175
RSDP-5-b	1044	278	28.5	6303	162	RSDPG-5-b	1138	287	28.5	6327	152
RSDP-5-s	1207	302	56.5	12280	143	RSDPG-5-s	1215	303	56.5	12282	141

B Detailed Comparison with the State-of-the-Art

B.1 Comparison with Hardware Designs

Table 11, Table 12 and Table 13 provide a detailed comparison of our design with previous literature including designs for AMD Artix-7 FPGAs for NIST security levels 1 and 2, 3, and 5. In all tables, the area is expressed in thousands of eSlices (KeSlice), while the efficiency indicator Area × Time (AT) product is expressed in KeSlices · ms. Table 11 is thus the basis for Figure 12.

Table 11: Synthesis results of KeyGen, Sign, and Verify for **security level 1 or 2**.

Design	Parameter Set	Resources					Freq.	KeyGen.		Sign		Verify	
		LUT	FF	BRAM	DSP	KeSlice	MHz	ms	AT	ms	AT	ms	AT
This work	RSDP-1-f	26741	11678	45.0	0	16.2	118	0.035	0.568	0.478	7.76	0.409	6.64
	RSDP-1-b	27308	11820	57.5	0	19.0	111	0.037	0.708	0.856	16.3	0.618	11.8
	RSDP-1-s	27779	11990	111.5	0	30.6	114	0.036	1.11	1.67	51.0	1.12	34.2
	RSDPG-1-f	27324	11726	28.5	0	12.9	122	0.008	0.109	0.338	4.35	0.303	3.91
	RSDPG-1-b	28572	11892	37.0	0	15.0	120	0.009	0.129	0.676	10.1	0.519	7.78
	RSDPG-1-s	28777	12257	63.0	0	20.6	117	0.009	0.182	1.38	28.4	1.03	21.1
[BNG24]	Dilithium-II	48600	30000	22.5	32	21.3	185	0.012	0.260	0.098	2.09	0.014	0.307
[WCW+24]	Dilithium-II	26521	8144	16	8	11.1	115	0.046	0.512	0.293	3.25	0.043	0.477
[LLL+24]	Dilithium-II	32320	9734	14	24	14.3	130	0.034	0.485	0.231	3.30	0.039	0.557
[ZZW+22]	Dilithium-II	29998	10366	11	10	11.2	96.9	0.043	0.481	0.290	3.24	0.046	0.514
[LSG21]	Dilithium-II	27433	10681	15.0	45	16.0	163	0.115	1.85	0.469	7.51	0.121	1.94
[HLL+25]	ML-DSA-44[3]	15370	7843	7.5	2	5.7	125	0.083	0.473	0.516	2.94	0.108	0.616
[ALCZ20]	SPHINCS+-128f	47991	72505	11.5	1	14.5	250	–	–	1.01	14.7	0.16	2.33
	SPHINCS+-128s	48231	72514	11.5	0	14.4	250	–	–	12.4	180	0.07	1.01
[DLK+25]	SPHINCS+-128f	10197	7357	8.0	0	4.25	150	–	–	19.3	81.9	1.91	8.10
	SPHINCS+-128s	10255	7410	8.0	0	4.26	150	–	–	363	1546	0.64	2.72
[OZZ+25]	Falcon-512	79065	46389	45.0	226	59.7	65	–	–	2.46	146	–	–
[BWMG23]	LESS-L1-b	54800	39900	59.5	0	26.3	200	0.145	3.82	26.0	684	25.7	678
	LESS-L1-i							0.387	10.2	25.6	674	25.4	670
	LESS-L1-s							0.872	22.9	20.8	548	20.6	544
[HSK+24]	MAYO-1[1]	92000	33000	45.5	2	32.9	75	0.400	13.1	1.28	42.1	0.400	13.1
	MAYO-1[2]	106000	38000	45.5	2	36.4	75	0.080	2.91	0.430	15.6	0.050	1.82
[DHSY24]	SDitH-L1-GF256	16592	8778	164.5	0	39.0	164	0.340	13.2	41.0	1601	52.9	2067
	SDitH-L1-GF251	17423	16336	164.5	196	65.5	164	0.350	22.9	130	8577	142	9360
[DLN+25]	MEDS-9923-b	24308	20514	112	66	38.7	116	–	–	65.2	1523	64.5	2496
	MEDS-13220-b	21680	19491	70	66	29.1	130	–	–	10.3	300	9.63	280
	MEDS-9923-hp	46842	44875	193	259	87.5	115	–	–	29.4	2573	30.0	2625
	MEDS-13220-hp	44080	43832	151	259	77.9	132	–	–	4.48	349	4.20	327
[NKZ+25]	Raccoon-128	13957	11284	30.5	4	10.5	222	0.472	4.96	1.26	13.3	0.436	4.58

[1] Corresponds to configuration 'e' of Table 5 in [HSK+24].
[2] Corresponds to configuration 'f' of Table 5 in [HSK+24].
[3] For the sake of comparison, we adjusted their *eSlice* computation to ours

Table 12: Synthesis results of KeyGen, Sign, and Verify for **security level 3**.

Design	Parameter Set	Resources					Freq.	KeyGen.		Sign		Verify	
		LUT	FF	BRAM	DSP	KeSlice	MHz	ms	AT	ms	AT	ms	AT
This work	RSDP-3-f	29096	13761	97.0	0	27.8	108	0.081	2.26	1.09	30.3	0.973	27.1
	RSDP-3-b	29790	14028	121.5	0	33.2	113	0.078	2.57	1.69	56.0	1.24	41.1
	RSDP-3-s	29742	14067	148.0	0	38.8	109	0.080	3.12	2.61	101	1.80	69.7
	RSDPG-3-f	31185	13272	43.5	0	17.0	112	0.017	0.297	0.701	11.9	0.665	11.3
	RSDPG-3-b	31323	13479	68.0	0	22.2	115	0.017	0.378	0.919	20.4	0.786	17.5
	RSDPG-3-s	32123	13808	122.5	0	34.0	118	0.017	0.564	1.63	57.2	1.35	45.7
[BNG24]	Dilithium-III	48600	30000	22.5	32	21.3	185	0.022	0.482	0.166	3.55	0.025	0.541
[WCW+24]	Dilithium-III	23378	9079	20	8	11.2	109	0.072	0.804	0.491	5.48	0.070	0.781
[LLL+24]	Dilithium-III	32320	9734	14	24	14.3	130	0.056	0.799	0.381	5.44	0.062	0.885
[ZZW+22]	Dilithium-III	29998	10366	11	10	11.2	96.9	0.060	0.671	0.461	4.65	0.064	0.715
[LSG21]	Dilithium-III	30900	11372	21.0	45	18.2	145	0.228	4.15	0.852	15.5	0.221	4.02
[HLL+25]	ML-DSA-65[4]	15370	7843	7.5	2	5.7	125	0.129	0.735	0.819	4.67	0.150	0.855
[ALCZ20]	SPHINCS+-192f	48398	73476	17.0	1	15.8	250	–	–	1.17	18.5	0.19	3.00
	SPHINCS+-192s	48725	72514	17.0	0	15.7	250	–	–	21.4	338	0.10	1.58
[DLK+25]	SPHINCS+-192f	10333	7590	8.0	0	4.28	150	–	–	31.1	133	2.76	11.8
	SPHINCS+-192s	10644	7643	8.0	0	4.36	150	–	–	631	2751	0.92	4.00
[BWMG23]	LESS-L3-b	76700	57900	102.5	0	40.9	167	0.432	17.7	235	9630	234	9607
	LESS-L3-s							0.796	32.5	277	11342	276	11324
[HSK+24]	MAYO-3[1,3]	147000	52000	96	2	57.3	100	0.650	37.2	2.15	123	0.670	38.4
	MAYO-3[2,3]	169000	60000	96	2	62.8	100	0.110	6.91	0.650	40.8	0.100	6.28
[DHSY24]	SDitH-L3-GF256	22569	13881	356	0	81.1	164	0.280	22.7	47.1	3826	66.8	5422
	SDitH-L3-GF251	29961	25794	356	382	134	164	0.290	38.9	149	20149	149	20149
[DLN+25]	MEDS-41711-b	34036	31243	206	130	69.7	123	–	–	79.8	5562	77.5	5402
	MEDS-69497-b	33178	30751	187	130	65.4	125	–	–	23.3	1524	20.2	1321
	MEDS-41711-hp	80525	91038	205	601	144	129	–	–	47.5	6840	46.2	6653
	MEDS-69497-hp	79653	90544	186	601	140	129	–	–	14.0	1960	12.2	1708
[NKZ+25]	Raccoon-192	13957	11284	30.5	4	10.5	222	0.739	7.76	1.89	19.8	0.682	7.16

[1] Corresponds to configuration 'e' of Table 5 in [HSK+24].
[2] Corresponds to configuration 'f' of Table 5 in [HSK+24].
[3] Numbers given for Kintex-7.
[4] For the sake of comparison, we adjusted their *eSlice* computation to ours

Table 13: Synthesis results of KeyGen, Sign, and Verify for **security level 5**.

Design	Parameter Set	Resources					Freq.	KeyGen.		Sign		Verify	
		LUT	FF	BRAM	DSP	KeSlice	MHz	ms	AT	ms	AT	ms	AT
This work	RSDP-5-f	32085	15537	151.0	0	40.0	113	0.138	5.51	1.71	68.4	1.60	64.1
	RSDP-5-b	33056	15635	202.0	0	51.1	111	0.140	7.16	2.73	140	2.09	107
	RSDP-5-s	34868	15815	280.0	0	68.1	105	0.148	10.1	4.62	314	3.27	223
	RSDPG-5-f	34626	15331	91.5	0	28.1	113	0.028	0.798	1.20	33.6	1.14	31.9
	RSDPG-5-b	36033	15502	122.5	0	35.0	113	0.028	0.995	1.57	54.9	1.32	46.1
	RSDPG-5-s	35296	15853	182.5	0	47.5	113	0.028	1.35	2.77	132	2.15	102
[BNG24]	Dilithium-V	48600	30000	22.5	32	21.3	185	0.030	0.646	0.196	4.18	0.033	0.708
[WCW+24]	Dilithium-V	28791	11338	24	8	13.4	114.3	0.109	1.46	0.632	8.44	0.103	1.38
[LLL+24]	Dilithium-V	32320	9734	14	24	14.3	130	0.084	1.20	0.425	6.07	0.098	1.40
[ZZW+22]	Dilithium-V	29998	10366	11	10	11.2	96.9	0.090	1.01	0.506	5.66	0.093	1.04
[LSG21]	Dilithium-V	44653	13814	31.0	45	23.7	140	0.363	8.63	1.040	24.6	0.377	8.96
[BNG21]	Dilithium-V	53187	28318	29.0	16	21.5	116	0.121	2.61	2.52	54.4	0.021	0.453
[HLL+25]	ML-DSA-87[4]	15370	7843	7.5	2	5.7	125	0.190	1.08	0.873	4.98	0.219	1.25
[ALCZ20]	SPHINCS+-256f	51009	74539	22.5	1	17.6	250	–	–	2.52	44.4	0.21	3.70
	SPHINCS+-256s	51130	74576	22.5	0	17.5	250	–	–	19.3	339	0.14	2.46
[DLK+25]	SPHINCS+-256f	10416	7672	8.0	0	4.30	150	–	–	61.7	265	2.81	12.0
	SPHINCS+-256s	10802	7698	8.0	0	4.40	150	–	–	557	2449	1.36	5.98
[BWMG23]	LESS-L5-b	104300	76700	167.5	0	61.5	143	0.941	57.9	909	55993	908	55924
	LESS-L5-s							1.73	106	610	37574	609	37511
[HSK+24]	MAYO-5[1,3]	215000	73000	194.5	2	95.2	100	1.19	113	3.89	370	1.19	113
	MAYO-5[2,3]	246000	83000	194.5	2	103	100	0.200	20.6	1.12	115	0.140	14.4
[DHSY24]	SDitH-L5-GF256	23323	14962	520.5	0	116	164	0.510	59.2	106	12384	152	17670
	SDitH-L5-GF251	34456	31409	521.5	472	182	164	0.530	96.8	276	50425	183	33530
[DLN+25]	MEDS-134180-b	38332	39570	285	163	91.9	130	–	–	86.7	7968	69.5	6387
	MEDS-167717-b	38011	39450	328	163	101	133	–	–	61.3	6191	40.0	4040
	MEDS-134180-hp	61595	60975	290	337	122	127	–	–	43.2	5270	36.0	4392
	MEDS-167717-hp	60620	60898	288	337	122	131	–	–	29.5	3599	20.6	2513
[NKZ+25]	Raccoon-256	13957	11284	30.5	4	10.5	222	1.20	12.6	2.93	30.8	1.11	11.7

[1] Corresponds to configuration 'e' of Table 5 in [HSK+24].
[2] Corresponds to configuration 'f' of Table 5 in [HSK+24].
[3] Numbers given for Kintex-7.
[4] For the sake of comparison, we adjusted their *eSlice* computation to ours

Figure 16 shows the full comparison when varying the security levels offered by the parameter sets. Note that the implementation for MAYO [HSK+24] implements the design for NIST category 3 and 5 on a Kintex-7 FPGA whose fabric is tailored to high performance, whereas the Artix-7 fabric is optimized for lower cost making a fair comparison more complicated. The topmost plots are given in the main document in Figure 12.

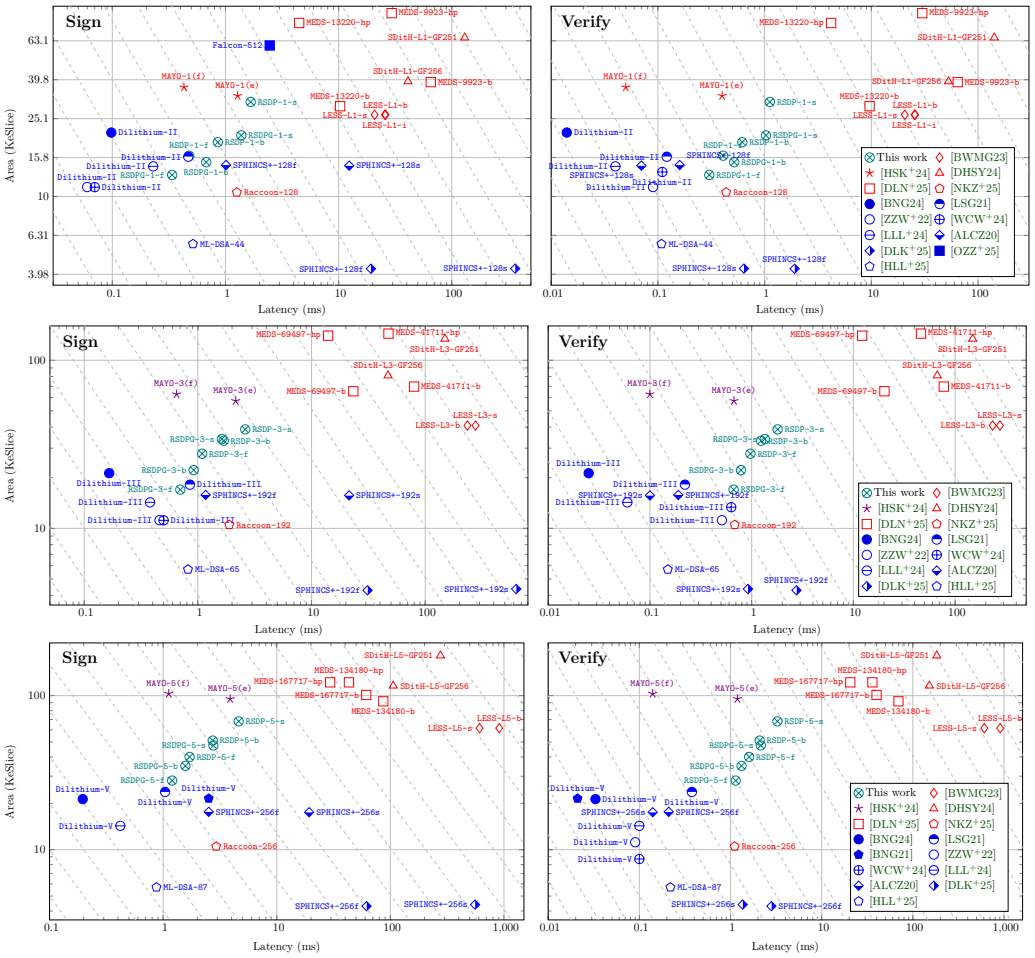


Figure 16: Performance and area results for SIGN and VERIFY primitives of hardware accelerators for post-quantum digital signatures of security levels 1 and 2 (top), 3 (mid) and 5 (bottom) evaluated on an Artix-7 FPGA, with the exception of marks colored in violet which are evaluated on a Kintex-7 FPGA. The work presented in this paper is highlighted in teal color. Blue points corresponds to designs of the standardized algorithms ML-KEM, FN-DSA, and SLH-DSA, while red points denote designs implementing the proposed algorithms in the on-ramp standardization effort. Plots use logarithmic scale axes, the dashed lines denote points in space with equal efficiency (closer to the origin is better, every step is a 2× improvement).

B.2 Comparison with Software Implementations

Table 14 compares our proposed design with performance numbers of optimized software implementation. The official round 2 specification [BBB+25a] (v2.2) reports cycle counts of an AVX2 optimized implementation running on an Intel i7-12700K processor clocked at 3.6GHz. In addition to that, we measured the reference implementation with disabled AVX2

and SSE extensions on an Intel i7-10700 processor running at 2.9GHz. For comparison we also provide numbers of an embedded implementation presented in [HMK+24, Table 4, 'light'] for an ARM Cortex-M4 running on a STM32L4R5ZI microcontroller that can run at up to 120MHz without introducing wait cycles by the flash. We note that this is the fastest embedded implementation of CROSS that runs on the target platform for all parameter sets we found, but still consumes considerable amount of memory. We also include the size optimized version from [SGB+25, Table 2] in the comparison, as it is the implementation with the lowest stack consumption for the STM32L4R5ZI in the literature so far.

Table 14: Comparison with software implementations.

	RSDP						RSDPG					
	KG		S		V		KG		S		V	
	kcc	μs	**kcc**	μs	**kcc**	μs	**kcc**	μs	**kcc**	μs	**kcc**	μs
This work – AMD Artix-7 (xc7a200tfbg484-3)												
1-f	4.13	35	56.3	476	48.3	409	1.03	8	41.2	338	37.0	303
1-b	4.13	37	95.0	856	68.6	618	1.03	9	81.1	676	62.3	519
1-s	4.13	36	190	1670	127	1120	1.03	9	162	1380	120	1030
3-f	8.76	81	118	1090	105	973	1.96	17	78.5	701	74.5	665
3-b	8.76	78	190	1690	140	1240	1.96	17	106	919	90.4	786
3-s	8.76	80	285	2610	196	1800	1.96	17	199	1630	159	1350
5-f	15.6	138	193	1710	181	1600	3.21	28	136	1200	129	1140
5-b	15.6	140	303	2730	232	2090	3.21	28	177	1570	149	1320
5-s	15.6	148	485	4620	343	3270	3.21	28	313	2770	243	2150
[BBB+25a, Table 8] – Intel i7-12700K @3.6GHz (opt, AVX2)												
1-f	52	14.4	1366	379	781	217	27	7.50	744	207	482	134
1-b	52	14.4	2361	656	1539	428	31	8.61	1452	428	989	275
1-s	51	14.2	4783	1329	3290	914	27	7.50	2849	791	1995	554
3-f	118	32.8	3110	864	1909	530	55	15.3	1745	485	1166	324
3-b	119	33.1	5099	1416	3500	972	56	15.6	2225	618	1508	419
3-s	119	33.1	7612	2114	5381	1495	55	15.3	4159	1155	3052	848
5-f	184	51.1	5501	1528	3453	959	94	26.1	2912	809	1961	545
5-b	184	51.1	8802	2445	6054	1682	94	26.1	3577	994	2463	684
5-s	183	50.8	14062	3906	10009	2780	92	25.6	6289	1747	4509	1253
Intel i7-10700 @2.9GHz (ref, no AVX2/SSE)												
1-f	76.6	26.4	5602	1932	2947	1016	36.7	12.7	2841	980	1851	638
1-b	76.8	26.5	9811	3383	3758	1296	36.9	12.7	5609	1934	3384	1167
1-s	76.7	26.4	19702	6794	6617	2282	36.5	12.5	11161	3849	6647	2292
3-f	179	61.7	17166	5919	8727	3009	76.2	26.3	7490	2583	4911	1693
3-b	181	62.4	28506	9830	9160	3159	76.9	26.5	9483	3270	5987	2064
3-s	181	62.4	42911	14797	11880	4097	76.6	26.4	17898	6172	10904	3760
5-f	310	107	36923	12732	18605	6416	129	44.5	15073	5198	10205	3519
5-b	314	108	59873	20646	17509	6038	129	44.5	18796	6481	11785	4064
5-s	310	107	97016	33454	22945	7912	129	44.5	33377	11509	19787	6823
	RSDP						RSDPG					
	KG		S		V		KG		S		V	
	Mcc	ms	**Mcc**	ms	**Mcc**	ms	**Mcc**	ms	**Mcc**	ms	**Mcc**	ms
[HMK+24, Table 4, 'light'] – ARM Cortex-M4 (STM32L4R5ZI) @120MHz												
1-f	0.438	3.65	17.0	142	9.12	76.0	0.200	1.67	11.3	94.0	6.19	51.6
1-b	0.438	3.65	34.2	285	17.5	145	0.200	1.67	26.0	217	13.8	115
1-s	0.438	3.65	68.5	571	35.5	296	0.200	1.67	51.5	430	27.7	231
3-f	1.02	8.52	41.9	349	22.8	190	0.445	3.71	24.8	207	14.3	119
3-b	1.02	8.52	75.3	628	37.2	310	0.445	3.71	36.3	302	20.0	166
3-s	1.02	8.52	114	946	55.6	463	0.445	3.71	68.6	571	39.1	326
5-f	1.82	15.2	75.3	628	41.3	344	0.744	6.20	44.1	368	25.6	213
5-b	1.82	15.2	130	1085	63.7	531	0.744	6.20	60.5	504	32.9	275
5-s	1.82	15.2	200	1663	99.6	830	0.744	6.20	108	898	59.0	492
[SGB+25, Table 2, 'Size Opt'] – ARM Cortex-M4 (STM32L4R5ZI) @120MHz												
1-f	–	–	37.7	314	10.4	86.9	–	–	21.9	182	6.65	55.4
1-b	–	–	74.7	622	18.2	152	–	–	51.1	426	14.0	117
1-s	–	–	151	1254	36.1	301	–	–	101	845	27.9	233
3-f	–	–	95.4	795	26.5	221	–	–	54.6	455	16.2	135
3-b	–	–	174	1447	39.5	329	–	–	79.0	658	21.7	180
3-s	–	–	262	2180	56.9	474	–	–	149	1243	41.2	344
5-f	–	–	221	1845	58.9	491	–	–	103	856	30.9	257
5-b	–	–	378	3154	73.1	609	–	–	141	1173	37.4	312
5-s	–	–	613	5106	108	898	–	–	252	2096	64.7	539